

Análisis del riesgo de ciberataques en vehículos conectados

CEUPE

MÁSTER BIG DATA ANALYTICS

TRABAJO FIN DE MÁSTER

Juliana Santacruz Restrepo

Análisis del riesgo de ciberataques en vehículos conectados

CEUPE

MÁSTER BIG DATA ANALYTICS

TRABAJO FIN DE MÁSTER

Juliana Santacruz Restrepo

Tutor:

Cristina Ruiz

David Quiñonez

2025

D./Dña: Juliana Santacruz Restrepo autoriza a que el presente trabajo se guarde y custodie en los repositorios de CEUPE.

Agradecimientos

Quiero expresar mi más profundo agradecimiento a mi familia, quienes han sido mi apoyo incondicional a lo largo de todo este proceso, su acompañamiento constante junto con sus palabras de ánimo han sido fundamentales para mantenerme motivada y superar cada desafío durante la realización de esta maestría.

En especial, agradezco profundamente a mis padres, quienes han estado a mi lado en todo momento. A mi madre, por su cariño y su apoyo constante, muy especialmente, a mi padre, por ser quien me motivó a dar este paso y me impulsó a iniciar esta maestría, la confianza de ellos en mí y su respaldo incondicional fueron claves para comenzar y continuar hasta el final.

Asimismo, deseo agradecer al programa por brindarme la oportunidad de aprender y profundizar en los conocimientos técnicos y conceptuales que hoy aplico. Gracias a esta formación, he desarrollado no solo habilidades profesionales sino también la perseverancia necesaria para afrontar y completar con éxito este proyecto.

Indice

Glosario.....	8
1. Introducción.....	10
2. Objetivos.....	12
2.1 Objetivo General.....	12
2.2 Objetivos Específicos.....	12
3. Metodología.....	13
3.1 Origen de los Datos.....	13
3.2 Preprocesamiento.....	14
3.2.1 Tecnologías por Utilizar.....	15
3.2.2 Integración de los Datos.....	15
3.3 Proceso Analítico.....	16
3.3.1 Entorno y Librerías.....	16
3.3.2 Importación de los Datos.....	17
3.3.3 Análisis de Datos.....	17
3.4 Modelamiento.....	18
4. Desarrollo e Implementación.....	19
4.1 Procesamiento y Carga.....	19
4.1.2 Transformación.....	22
4.1.3 Etiquetado.....	23
4.1.4 Exportación del Dataset Unificado.....	23
4.2 Análisis de Datos.....	24
4.2.1 Análisis Exploratorio de Datos (EDA).....	24
4.2.1.1 Estructura del Conjunto de Datos.....	25
4.2.1.2 Distribución de Variables.....	26
4.2.1.3 Visualización.....	27
4.3 Modelado.....	35
4.3.1 Preparación y Limpieza de datos.....	36
4.3.2 Desarrollo del Modelo.....	37
4.3.2.1 Modelo Supervisado.....	45
4.3.2.2 Modelo No Supervisado.....	47
4.3.2.3 Pipeline Híbrido.....	47
4.3.3 Visualizaciones.....	50

5. Recomendaciones para la Mejora de la Ciberseguridad Vehicular.....	56
6. Conclusiones.....	57
7. Recomendaciones y Prospectiva.....	58
8. Referencia Bibliográfica.....	60

Índice de Figuras

Fig 1. Diferencias en los formatos de los archivos de entrada.....	20
Fig 2. Sesión y carga de los archivos independientes.....	20
Fig 3. Asociación entre archivos de entrada y su tipo de ataque.....	21
Fig 4. Función de integración y procesamiento de archivos CSV, TXT.....	22
Fig 5. Integración de los archivos procesados.....	22
Fig 6. Transformación de campos con tipo de dato hexadecimal a entero.....	23
Fig 7. Proceso de etiquetado multiclase y binario.....	23
Fig 8. Esquema del conjunto de datos.....	25
Fig 9. Frecuencia de registros por tipo de señales normales o ataques.....	28
Fig 10. Distribución temporal de señales vehiculares normales y de ataque.....	29
Fig 11. Frecuencia de mensajes según identificador CAN y tipo de señal vehicular.....	29
Fig 12. Distribución del tamaño de los mensajes (DLC) por tipo de señal vehicular.....	30
Fig 13. Valores promedio de los bytes en mensajes CAN según el tipo de señal vehicular.....	31
Fig 14. Distribución de mensajes legítimos y manipulados según tipo de señal vehicular.....	31
Fig 15. Distribución del identificador CAN por tipo de señal vehicular.....	32
Fig 16. Matriz de correlación entre variables numéricas.....	33
Fig 17. Frecuencia de los 20 identificadores CAN más comunes en la red vehicular.....	34
Fig 18. Distribución de entropía en la red CAN según el tipo de señal vehicular.....	34
Fig 19. Limpieza e imputación de valores nulos.....	36
Fig 20. Reducción del dataframe.....	36
Fig 21. Esquema del conjunto de datos para el modelado.....	37
Fig 22. Selección de variables objetivo modelo binario y multiclase.....	40
Fig 23. División estratificada y temporal para los modelos supervisados.....	41
Fig 24. Balanceo de clases clasificación binaria.....	42
Fig 25. Balanceo SMOTE y codificación de etiquetas para la clasificación multiclase.....	42
Fig 26. Ajuste dinámico de pesos por clase para la clasificación multiclase.....	42
Fig 27. Escalado para la clasificación binaria.....	43
Fig 28. Escalado e imputación para la clasificación multiclase.....	43
Fig 29. Creación y entrenamiento del modelo binario.....	45
Fig 30. Creación y entrenamiento del modelo multiclase.....	46
Fig 31. Creación y entrenamiento del modelo Isolation Forest.....	47
Fig 32. Creación y entrenamiento del modelo Autoencoder Denso.....	47

Fig 33. Implementación de pipeline híbrido con lógica de umbral dinámico.....	48
Fig 34. Mapa de calor de correlación entre variables del bus CAN.....	50
Fig 35. Ranking de las 15 características más relevantes según el modelo binario.....	51
Fig 36. Comparación de métricas de desempeño entre modelos de detección.....	51
Fig 37. Comparación integral del desempeño de modelos de detección.....	52
Fig 38. Desempeño del modelo binario en detección de ataques.....	53
Fig 39. Desempeño del modelo Autoencoder Denso en detección de ataques.....	53
Fig 40. Matriz de confusión normalizada del modelo pipeline híbrido multiclase.....	54

Indice de Tablas

Tabla 1. Descripción detallada de los tipos de datos dentro del dataset.....	14
Tabla 2. Detalle atributos del dataset.....	14
Tabla 3. Diferencias de los atributos y los tipos de datos entre datasets.....	24
Tabla 4. Cantidad de valores nulos y porcentaje en la columna encontrada.....	26
Tabla 5. Cantidad de valores únicos.....	26
Tabla 6. Distribución estadística.....	27
Tabla 7. Balance del dataset.....	27
Tabla 8. Descripción de las variables usadas para la ingeniería de características.....	40
Tabla 9. Comparación de técnicas de balanceo y tiempos de ejecución.....	42
Tabla 10. Comparativa técnica entre las versiones de modelado.....	45
Tabla 11. Resumen comparativo del desarrollo de modelos según versión de desarrollo.....	49
Tabla 12. Comparación de métricas de rendimiento de modelos de detección.....	56

Glosario

BLE: (Por sus siglas en inglés, Bluetooth Low Energy). Es una tecnología inalámbrica diseñada para aplicaciones que requieren bajo consumo energético y una transmisión de datos eficiente a corta distancia, es utilizada en dispositivos IoT, sistemas de domótica, wearables (dispositivos electrónicos que se pueden llevar puestos como relojes) y automóviles, especialmente para funciones como llaves digitales, conexión con smartphones o sensores.

CAN: (Por sus siglas en inglés, Controller Area Network). Es el sistema de comunicación interno de vehículos que permite la transmisión de datos entre diferentes unidades de control electrónico (ECUs) sin necesidad de un equipo central. En este caso, permite la automatización de funciones tales como frenado, la transmisión o la dirección asistida del vehículo.

Dataset: Es un conjunto estructurado de datos recopilados y organizados con el propósito de ser analizados, modelados o utilizados en procesos de investigación.

DCL: (Por sus siglas en inglés, Data Length Code). Es un campo presente en los mensajes transmitidos por la red CAN que indica cuántos bytes de datos contiene el mensaje, su valor puede variar entre 0 y 8, dependiendo del contenido transmitido. Esta información es importante para interpretar correctamente la estructura del mensaje y procesar los datos asociados de manera adecuada.

DoS: (Por sus siglas en inglés, Denial of Service). Es un tipo de ciberataque, el cual su propósito es interrumpir el funcionamiento normal de un sistema sobrecargando el sistema con señales o peticiones maliciosas y repetitivas que impiden la comunicación de los dispositivos legítimos.

ECU: (Por sus siglas en inglés, Electronic Control Unit). Es un dispositivo electrónico el cual está integrado en los vehículos, se encarga de controlar funciones específicas como el motor, frenos, sistema de dirección, entre otros. Este dispositivo es de suma importancia, ya que recopila información de sensores, procesa datos y envía datos a otros componentes del vehículo para garantizar el funcionamiento adecuado del mismo.

EDA: (Por sus siglas en inglés, Exploratory Data Analysis). Es el proceso inicial para examinar, resumir y visualizar los datos disponibles con el objetivo de identificar patrones, detectar valores atípicos, analizar distribuciones y entender la estructura general del dataset antes de aplicar modelos predictivos.

Fuzzing attack: Es un tipo de ataque de seguridad donde se testea el envío de datos de manera aleatoria o errónea con el fin de detectar vulnerabilidades/bugs en el software, firmware, redes, hasta en hardware.

Relay Attack: Es una técnica de ciberataque que intercepta y retransmite señales inalámbricas entre un dispositivo legítimo, como una llave digital, y el sistema que desea autenticarse como un vehículo, con el objetivo de engañar al receptor y simular una proximidad falsa.

Rolling Windows: Es una técnica que se usa para analizar datos en series de tiempo o secuencias, el cual consiste en mirar grupos pequeños y consecutivos de datos, llamados “ventanas” que se mueven paso a paso a lo largo de toda la serie. Esto permite observar cómo cambian las características o resultados con el tiempo, en lugar de analizar toda la información de una sola vez.

Spoofing/Manipulación: Es una técnica implementada por ciberdelincuentes donde manipulan mensajes o direcciones dentro del sistema, provocando que el usuario reciba información falsa como si proviniera de una fuente confiable. Además, permite alterar el comportamiento de sensores, ECUs o comandos críticos del vehículo.

SMOTE: (Por sus siglas en inglés, Synthetic Minority Over-sampling Technique). Es una técnica de sobremuestreo que genera nuevas instancias sintéticas para clases minoritarias, con el fin de equilibrar conjuntos de datos desbalanceados y mejorar el rendimiento de modelos de clasificación.

SMOTEEN: (Por sus siglas en inglés, Synthetic Minority Over-sampling Technique Edited Nearest Neighbours). Es una técnica híbrida utilizada en el preprocesamiento de datos desbalanceados en aprendizaje automático, el cual combina el método SMOTE con ENN (Edited Nearest Neighbours), que elimina instancias ruidosas o mal clasificadas de la clase mayoritaria. Su objetivo es mejorar el rendimiento de los modelos de clasificación al proporcionar un conjunto de datos más equilibrado y limpio.

UDF: (Por sus siglas en inglés, User Defined Function). Es una función personalizada creada por el usuario para realizar operaciones específicas que no están cubiertas por las funciones predefinidas del entorno o del lenguaje de programación.

1. Introducción

La industria automotriz ha experimentado una transformación profunda impulsada por la integración de tecnologías digitales avanzadas, esta evolución no solo ha dado paso a la popularización de los vehículos eléctricos, donde empresas como Tesla han marcado un precedente, sino que también ha consolidado un nuevo paradigma donde los vehículos se configuran como plataformas tecnológicas sobre ruedas donde implementan elementos como pantallas táctiles, conectividad a internet, sistemas de navegación inteligentes y sensores interconectados lo que refleja una integración cada vez más compleja entre software y hardware.

Este crecimiento tecnológico ha traído consigo una arquitectura electrónica más sofisticada, basada en múltiples unidades de control electrónico (ECUs), redes de comunicación internas como el protocolo CAN y sistemas distribuidos que gestionan funciones desde las más básicas hasta operaciones críticas de seguridad y conducción autónoma; Sin embargo, esta digitalización no está exenta de vulnerabilidades las cuales se han ampliado notablemente en ataques para potenciales ciberamenazas, afectando directamente la seguridad vial y del usuario.

Es por eso que entre más vehículos conectados a redes inalámbricas, aumentan también los riesgos asociados a ciberataques, de manera que amenazas como fuzzing, spoofing o denegación de servicios (DoS) han dejado de ser casos hipotéticos para convertirse en peligros tangibles. De hecho, en 2022, investigadores de la firma NCC Group demostraron cómo era posible desbloquear y arrancar un Tesla Model 3 o Model Y mediante un relay attack, explotando vulnerabilidades en el protocolo Bluetooth Low Energy (BLE), utilizado para las llaves digitales. Este tipo de ataque, que no requiere acceso físico al vehículo, expuso las limitaciones de los sistemas de autenticación inalámbrica. Debido a lo anterior, queda evidenciado lo vulnerable que puede llegar a ser un sistema que, paradójicamente, fue diseñado para ofrecer mayor seguridad y control.

En referencia al tema se vuelve imprescindible el desarrollo de estrategias y soluciones que permitan detectar, anticipar y mitigar posibles intrusiones informáticas en el campo vehicular, ya que la proyección referente al aumento masivo de vehículos conectados en años futuros presenta un desafío urgente, no solo desde una perspectiva técnica y comercial, sino también desde el punto de vista de la seguridad pública y confianza del consumidor. Por otro lado, la protección de los sistemas internos del vehículo ya no puede abordarse exclusivamente desde la ingeniería mecánica o electrónica, sino que requiere una mirada interdisciplinaria donde el análisis de datos, la inteligencia artificial y la ciberseguridad juegan un papel fundamental.

Por lo tanto, el presente trabajo se enfoca en el análisis del riesgo asociado a ciberataques en vehículos conectados, utilizando el Car Hacking Dataset, desarrollado por el laboratorio OCSLab, este conjunto de datos incluye simulaciones de ataques reales sobre el bus CAN, como fuzzing,

spoofing y DoS, así como registros en condiciones normales de operación. El desarrollo se aborda desde una perspectiva analítica, basada en técnicas escalables de procesamiento de datos y en el uso de modelos de machine learning supervisado y no supervisado, con el objetivo de contribuir al diseño de sistemas de detección temprana que fortalezcan la seguridad de los vehículos conectados y permitan anticipar amenazas en entornos reales.

2. Objetivos

2.1 Objetivo General

Analizar el riesgo de ciberataques en vehículos conectados mediante el uso de datos simulados del bus CAN, con el propósito de detectar ataques como spoofing, DoS y Fuzzing, y aportar al desarrollo de estrategias eficaces para su detección y prevención a través del modelado de datos y técnicas de aprendizaje automático.

2.2 Objetivos Específicos

- Procesar y transformar los datos del conjunto Car-Hacking Dataset, aplicando técnicas de limpieza y tratamiento de valores atípicos o nulos, con el fin de asegurar la calidad del dato y su aptitud para análisis posteriores.
- Realizar un análisis exploratorio de los datos (EDA) que permita identificar patrones de comportamiento normales y anómalos en las señales vehiculares, mediante visualizaciones descriptivas y estadísticas que caracterizan los ataques de tipo spoofing, DoS y fuzzing.
- Diseñar y entrenar modelos de aprendizaje automático, tanto supervisado como no supervisado, con el fin de identificar comportamientos anómalos y patrones asociados a ciberataques en el bus CAN, contribuyendo a la detección temprana de amenazas en vehículos conectados.
- Proponer recomendaciones para la mejora de los sistemas de ciberseguridad en vehículos conectados, basadas en la evidencia obtenida a partir del análisis de datos y la detección temprana de ciberataques simulados.

3. Metodología

Mediante un enfoque analítico se dará desarrollo al proyecto aplicado a la seguridad vehicular, implementando como fuente de información el dataset “Car Hacking Dataset”, que contiene registros de tráfico CAN en condiciones normales y bajo distintos tipos de ciberataques simulados como fuzzing, spoofing y DoS. El propósito es detectar los ataques anteriormente mencionados mediante técnicas de análisis de datos y algoritmos de machine learning.

De acuerdo a lo anterior, se propone un pipeline compuesto por cuatro etapas principales:

- a. Origen de los datos.
- b. Preprocesamiento (Tecnologías a implementar e integración de los datos).
- c. Análisis exploratorio de los datos (EDA).
- d. Modelado (Aprendizaje automático supervisado y no supervisado).

3.1 Origen de los Datos

Se ha seleccionado el conjunto de datos Car-Hacking Dataset, desarrollado por el laboratorio de investigación en ciberseguridad OCSLab. Este recurso se encuentra disponible públicamente en el siguiente enlace oficial [Car-Hacking Dataset](#).

Este dataset tiene como finalidad facilitar investigaciones orientadas a la detección de ataques sobre redes internas de vehículos, en particular sobre el bus CAN. Este contiene tanto registros de condiciones normales de operación como simulaciones de distintos tipos de ciberataques como fuzzing, spoofing y DoS.

A continuación, se presenta a detalle los archivos dentro del dataset los cuales son independientes y representan distintos tipos de tráfico CAN.

Archivo / Carpeta	Tipo de ataque / función	Descripción breve
Fuzzy_dataset.csv	Fuzzing attack	Datos de inyecciones aleatorias de mensajes CAN para simular ataques de tipo fuzzing.
DoS_dataset.csv	Denegación de servicios (DoS)	Simula sobrecarga del bus CAN, bloqueando la comunicación normal.
gear_dataset.csv	Spoofing / Manipulación	Alteración de señales relacionadas con el engranaje del vehículo.

Archivo / Carpeta	Tipo de ataque / función	Descripción breve
RPM_dataset.csv	Spoofing / Manipulación	Manipulación de la señal de revoluciones por minuto del motor.
normal_run_data.txt	Sin ataque (condición normal)	Datos de referencia sin interferencias maliciosas, útiles para comparar con los ataques.

Tabla 1. Descripción detallada de los tipos de datos dentro del dataset. Fuente: Autor.

Por otro lado, cada archivo del dataset comparte una estructura común de columnas, las cuales corresponden al formato típico del tráfico CAN. A continuación, se presenta a detalle los atributos presentes en cada archivo del dataset.

Atributos del dataset				
Timestamp	CAN ID	DLC	Data [0] a Data [7]	Flag
Indica el momento exacto en que fue registrado el mensaje, expresado en segundos.	Es el identificador del mensaje en formato hexadecimal (ej: 043f), el cual determina el tipo de dato o la unidad emisora dentro del sistema.	Especifica el número de bytes de datos contenidos en el mensaje, con un rango entre 0 a 8.	Representa los valores de los datos del mensaje CAN dentro de un formato byte de 8 bits. Dependiendo del tipo de mensaje pueden tener un significado específico referente a velocidad, RPM, engranajes, etc.	Es un campo categórico que indica el tiempo del mensaje según su origen. Tiene 2 tipos de valores: a. R (Regular): Mensaje legítimo o normal. b. T (Tampered): Mensaje manipulado o inyectado artificialmente para simular un ataque.

Tabla 2. Detalle atributos del dataset. Fuente: Autor.

3.2 Preprocesamiento

Esta etapa es crucial dentro del desarrollo del proyecto, debido a que la información con la que se trabaja en este caso datos complejos, masivos y heterogéneos deben pasar por un proceso de

integración, limpieza, transformación y estructuración de las variables para posteriormente alimentar los modelos de machine learning. Los archivos pertenecientes al dataset elegido cuentan con millones de registros distribuidos en 5 diferentes archivos donde simulan datos normales o tipos de ataques, es por eso que se ha empleado un enfoque basado en tecnologías de procesamiento distribuido con técnicas específicas para el manejo de series temporales, codificación de mensajes, detección de anomalías y normalización de variables.

3.2.1 Tecnologías por Utilizar

Para el desarrollo del proyecto se implementarán diferentes herramientas tecnológicas, las cuales permitirán abordar de manera eficiente y escalable el proceso de carga, transformación y análisis de grandes volúmenes de datos, además de la implementación en modelos de machine learning orientados a la detección de ciberataques. La selección de las tecnologías se basó en la capacidad para gestionar el procesamiento masivo de datos, análisis estadístico, visualización y modelos de predicción, conforme a los conocimientos adquiridos a lo largo del máster.

- **Entorno de desarrollo:** Se utilizará Jupyter Notebook como el entorno principal para la implementación de todo el proceso de codificación y se implementará Apache Spark para el procesamiento distribuido de los datos, ya que se cuenta con un dataset de gran tamaño.
- **Librerías:** Se implementan diferentes tipos de librerías en su mayoría relacionadas a spark como SparkSession para gestionar una sesión de Spark para poder trabajar con estructuras de datos distribuidos en PySpark, pyspark.sql.functions para la manipulación de columnas, creación de condiciones lógicas transformación de datos, entre otros, también, pyspark.sql.types.IntegerType para definir y transformar tipos de datos, particularmente convertir columnas a formato entero durante la limpieza.

3.2.2 Integración de los Datos

Debido al gran tamaño de los archivos y la cantidad de registros dentro de ellos, se implementará Apache Spark dentro de Jupyter Notebook para realizar el procesamiento distribuido, con el fin de manejar los datos de manera eficiente siendo posible el filtrar, limpiar y transformar sin comprometer la memoria local del entorno y aplicando lo visto durante el máster. A continuación se presentan las fases que se llevarán a cabo:

- a. **Carga e integración:** Se realizará una lectura distribuida de los cuatro archivos CSV y un archivo tipo texto con PySpark, posteriormente se unificarán en un único dataframe general utilizando el 30% de los registros de cada archivo, realizando la verificación de que no se encuentren registros duplicados o corruptos.

- b. Transformación:** Se hará la conversión de aquellos campos que contengan valores hexadecimales (CAN ID y Data[0] a Data[7]) a enteros (CAN_ID_dec y DATA_0_int a DATA_7_int), con el fin de facilitar el análisis exploratorio y el modelado.
- c. Etiquetado:** Se implementará la clasificación multiclase llamada Tipo_ataque que identificará a que archivo cargado corresponde, en este caso, si es normal o ataque (fuzzing, spoofing o DoS). Además, se derivará una etiqueta binaria para implementarla en los modelos referentes a la clasificación binaria y detección de anomalías, esta se identificará como Ataque:True/False.

3.3 Proceso Analítico

Esta etapa permite conocer en profundidad la estructura, comportamiento y relación del conjunto de datos a través de un análisis exploratorio, donde se identifican patrones, anomalías, posibles hipótesis y líneas de investigación las cuales guiarán la línea de trabajo no solo para la parte analítica, sino también la parte de modelamiento y recomendaciones de mejora para evitar ataques a vehículos conectados.

3.3.1 Entorno y Librerías

Para abordar el desarrollo, se estableció un entorno de análisis robusto basado en PySpark y bibliotecas tanto estadísticas como visuales de Python. A continuación se detallan las herramientas a emplear:

- a. Pyspark:** Se utilizará para crear un entorno que permite trabajar con grandes volúmenes de datos de forma distribuida, ideal para manejar el conjunto de datos implementado que cuenta con más de 17 millones de registros. Dentro de esta herramienta, se usaron componentes como pyspark.sql para gestionar los datos, pyspark.sql.functions para aplicar funciones a las columnas, y pyspark.sql.types para definir los tipos de datos.
- b. Spark MLlib:** Se empleará la herramienta StringIndexer para convertir variables categóricas en números, lo cual es necesario antes de aplicar ciertos métodos de análisis.
- c. Pandas y NumPy:** Estas librerías ayudarán al análisis y la manipulación de datos más pequeños o resultados derivados, aprovechando su eficiencia para trabajar en memoria.
- d. Matplotlib y Seaborn:** Serán las principales herramientas para creación de gráficos y visualización de datos, empleadas para mostrar distribuciones, relaciones, tendencias a lo largo del tiempo y detectar valores atípicos mediante gráficos como barras, diagramas de violín, box plots, gráficos agrupados, entre otros.

- e. **Scikit-learn:** Se aplicarán técnicas de estandarización de variables numéricas para facilitar el análisis posterior con StandardScaler.
- f. **Otros recursos:** También se utilizarán herramientas adicionales como collections.Counter, warnings y math para tareas complementarias como conteos, control de advertencias y cálculos matemáticos.

3.3.2 Importación de los Datos

El conjunto de datos será cargado en formato CSV utilizando una sesión de Spark, que permite subir eficientemente los 17.558.347 registros, también se realizará la verificación del conjunto de datos cargado mediante el esquema del dataset donde se revisarán los tipos de datos por columna, su consistencia con el contenido real y aquellos campos de tipo numérico que se marcan como string debido a su formato hexadecimal.

3.3.3 Análisis de Datos

Se llevará a cabo un análisis exploratorio de los datos (EDA) con el fin de comprender la naturaleza del conjunto de datos y detectar posibles inconsistencias, patrones o aspectos relevantes que puedan influir en las etapas posteriores del proyecto.

Este análisis se desarrollará en tres bloques principales, que permitirán abordar de forma estructurada la calidad, distribución y comportamiento de las variables tanto numéricas como categóricas. De acuerdo a lo anterior el EDA se organizará de la siguiente manera:

- a. **Estructura del conjunto de datos:** Se analizará la organización general del conjunto de datos para corroborar su integridad y calidad, donde se hallarán posibles falencias en el formato o en datos redundantes que requieran de una posterior transformación. Por lo tanto, se evaluará lo siguiente:
 - El esquema general del dataset (columnas y tipos de datos).
 - La cantidad y proporción de valores nulos por columna.
 - La cantidad de valores únicos por columna.
 - La presencia de columnas con valores constantes (sin variación), ya que no aportan información útil al análisis.
- b. **Distribución de variables:** Se analizarán las características estadísticas de las variables con el propósito de anticipar sesgos, desbalances o variabilidad que podrían afectar etapas posteriores. Para esta fase se validará lo siguiente:

- Descripción estadística de variables numéricas (media, mediana, percentiles, desviación estándar, etc.).
 - Clasificación de variables categóricas y numéricas.
 - Análisis del balance de clases en la variable Ataque.
- c. **Visualización de datos:** Se emplearán diversas técnicas de visualización para representar gráficamente el comportamiento del conjunto de datos, facilitando el hallazgo de patrones, relaciones y valores atípicos. En esta etapa se incluirán:
- Gráficos de barras para observar frecuencias absolutas y relativas.
 - Boxplots para analizar la distribución y presencia de outliers.
 - Gráficos de correlación para evaluar la relación entre variables numéricas (matriz de correlación con mapa de calor).
 - Gráficos de violín para visualizar la dispersión de ciertas variables categóricas.
 - Análisis temporal mediante gráficos de series para detectar comportamientos a lo largo del tiempo.

3.4 Modelamiento

El proceso de modelado estará organizado por diferentes etapas estructuradas, que se desarrollarán con el propósito de diseñar, entrenar y evaluar modelos de aprendizaje automático que sean capaces de detectar y clasificar diferentes tipos de ciberataques en vehículos conectados. A continuación se describen las fases metodológicas previstas para el desarrollo:

- **Entorno y librerías:** Se usará el entorno de trabajo Jupyter Notebook como el entorno principal para la implementación de todo el proceso de codificación desde la limpieza, hasta el desarrollo de los modelos, se emplearán librerías de procesamiento distribuido de Apache Spark para la carga y procesamiento eficiente de los datos junto con librerías de análisis y modelado como pandas, scikit-learn, XGBoost, imbalanced-learn, y herramientas de visualización como matplotlib y seaborn.
- **Preparación y limpieza de datos:** Se realizará la carga del conjunto de datos y según los hallazgos encontrados en la fase de análisis exploratorio de datos se desarrollará la eliminación o imputación de valores nulos en campos numéricos (imputación con valores estadísticos), el relleno o filtrado de valores faltantes en variables categóricas relevantes y la eliminación de registros incompletos o que no presenten una importancia para incluir en el modelo.

- **Modelo supervisado:** El proceso de modelado se dividirá en dos fases complementarias: un modelo binario, el cual consta de entrenar un clasificador para detectar si una muestra corresponde a un tráfico normal o a un ataque y un modelo multiclase, el cual se entrenará para que identifique el tipo específico de ataque (Spoofing, DoS o Fuzzing).
- **Modelado no supervisado:** Se entrenará basado en Isolation Forest, con el objetivo de detectar posibles anomalías en el tráfico que no estén previamente etiquetadas, con el fin de detectar casos de ataques desconocidos.
- **Visualización de resultados:** Se emplearán visualizaciones para facilitar la interpretación de los modelos.

4. Desarrollo e Implementación

En esta sección se presenta el desarrollo práctico del proyecto, acompañado de visualizaciones, fragmentos de código e imágenes que ilustran los resultados obtenidos, la estructura de este apartado sigue el orden propuesto en la metodología, con la excepción del apartado correspondiente al origen de los datos, que ya ha sido explicado anteriormente.

Se muestra el proceso completo de preparación, análisis, transformación de los datos y modelado, con el objetivo de abordar una problemática actual relacionada con la seguridad en entornos vehiculares inteligentes. De esta manera, cada apartado refleja el enfoque analítico propuesto, y a su vez la aplicación de los conocimientos adquiridos respecto a técnicas de análisis de datos, machine learning y visualización de forma estructurada.

4.1 Procesamiento y Carga

Se lleva a cabo la carga inicial y el preprocesamiento estructurado de los datos provenientes de cinco archivos independientes relacionados con ataques y tráfico normal en sistemas de vehículos conectados, los cuales antes de realizar el proceso de unificación, se verifica el estado de los registros y archivos como su formato, campos faltantes, entre otros elementos. Es importante aclarar que, si bien en el objetivo específico se contempla el tratamiento de valores atípicos o nulos en esta etapa, se decidió posponer dicha limpieza para una fase posterior, con el fin de observar y analizar ese comportamiento durante el análisis exploratorio de los datos.

De esta manera, se presentan en distintos tipos de archivos, cuatro en formato CSV y uno en formato TXT, de ellos, solo el archivo referente al tráfico normal (TXT) contenía los campos especificados, en cambio, los archivos de información sobre ataques (CSV) no incluían encabezados, siendo necesario asignar manualmente las columnas correspondientes a los atributos iniciales (Timestamp, CAN ID, DLC, DATA[0] a DATA[7] y Flag).

Por otro lado, como se indica en la Figura 1, ambos tipos de archivo presentaban estructuras diferentes y una discrepancia en cuanto a la cantidad de campos, el archivo de tráfico normal (imagen superior) no incluía el campo Flag, pero contaba con campos delimitados y estructurados, de manera contraria, en el archivo de ataque (imagen inferior) se encuentran los registros sin encabezados, se observa que contienen el campo Flag y los datos están dispuestos en un formato separado por comas.

Timestamp:	1479121434.850202	ID: 0350	000	DLC: 8	05 28 84 66 6d 00 00 a2
Timestamp:	1479121434.850423	ID: 02c0	000	DLC: 8	14 00 00 00 00 00 00 00
Timestamp:	1479121434.850977	ID: 0430	000	DLC: 8	00 00 00 00 00 00 00 00
Timestamp:	1479121434.851215	ID: 04b1	000	DLC: 8	00 00 00 00 00 00 00 00
Timestamp:	1479121434.851463	ID: 01f1	000	DLC: 8	00 00 00 00 00 00 00 00

A1		1478195721.903877,0545,8,d8,00,00,8a,00,00,00,00,R					
	A	B	C	D	E	F	G
1	1478195721.903877,0545,8,d8,00,00,8a,00,00,00,00,R						
2	1478195721.905736,02b0,5,ff,7f,00,05,49,R						
3	1478195721.908437,0002,8,00,00,00,00,00,01,07,15,R						
4	1478195721.908676,0153,8,00,21,10,ff,00,ff,00,00,R						
5	1478195721.909414,0130,8,19,80,00,ff,fe,7f,07,60,R						

Fig 1. Diferencias en los formatos de los archivos de entrada. Fuente: Autor.

De acuerdo a los hallazgos identificados anteriormente en un análisis inicial, se procede a cargar la información para realizar la asociación, transformación, etiquetado y la unificación de los archivos, este proceso se llevó a cabo de la siguiente manera:

1. Carga y asociación de archivos:

Se preparó el entorno en el que se iba a desarrollar todo el proceso de la unificación de los archivos los cuales se encontraban en memoria local, de esta manera se debía cargar al entorno de trabajo en este caso Jupyter Notebook para el manejo de datos con PySpark.

```
# Crear sesión Spark
spark = SparkSession.builder \
    .appName("Car Hacking Dataset") \
    .getOrCreate()

#Lectura de los 5 archivos cargados
df_car_hacking = spark.read.csv("/content/Fuzzy_dataset.csv", header=False, inferSchema=False)
df_car_hacking = spark.read.csv("/content/DoS_dataset.csv", header=False, inferSchema=False)
df_car_hacking = spark.read.csv("/content/gear_dataset.csv", header=False, inferSchema=False)
df_car_hacking = spark.read.csv("/content/RPM_dataset.csv", header=False, inferSchema=False)
df_car_hacking = spark.read.csv("/content/normal_run_data.txt", header=False, inferSchema=False)
```

Fig 2. Sesión y carga de los archivos independientes. Fuente: Autor.

A su vez, para facilitar la identificación y posterior etiquetado de datos, se estableció un diccionario que asocia a cada archivo con su respectivo tipo de tráfico, ya sea normal o un ataque en específico.

```
# Diccionario archivo -> tipo de ataque
archivo_tipo = {
    '/content/Fuzzy_dataset.csv': 'Fuzzing_Attack',
    '/content/DoS_dataset.csv': 'DoS_Attack',
    '/content/gear_dataset.csv': 'Spoofing_Gear_Attack',
    '/content/RPM_dataset.csv': 'Spoofing_RPM_Attack',
    '/content/normal_run_data.txt': 'Normal_Traffic'
}
```

Fig 3. Asociación entre archivos de entrada y su tipo de ataque. Fuente: Autor.

2. Función de carga e integración de archivos:

Para manejar la lectura y transformación de los archivos de entrada se desarrolló una función llamada “carga_integracion”, que tiene como parámetros el tipo de archivo, tipo de ataque asociado, y un porcentaje de muestreo (30%), en este caso, se eligió ese porcentaje para reducir la cantidad de datos cargados por archivo y evitar la saturación del entorno de desarrollo. Para cada formato se estipuló lo siguiente:

- a. **Formato csv:** Se debe realizar la lectura sin cabecera (ya que no cuentan con una), se asigna manualmente las columnas y se convierten los campos Timestamp y DLC a tipo de dato double e int respectivamente.
- b. **Formato txt:** Se debe realizar la lectura línea por línea del contenido del archivo, se emplean expresiones regulares para extraer los campos, se deben convertir los tipos de dato de los campos Timestamp y DLC al igual que en el anterior formato y se establece un valor por defecto en el campo Flag en este caso R que refiere a un mensaje legítimo o normal.

```

def carga_integracion(archivo, tipo_ataque, porcentaje=0.3):
    if archivo.endswith(".csv"):
        # Lee los archivos CSV
        df_car_hacking = spark.read.csv(archivo, header=False, inferSchema=False)

        #Estipula las columnas
        columnas = ["Timestamp", "CAN ID", "DLC", "DATA[0]", "DATA[1]", "DATA[2]", "DATA[3]",
                    "DATA[4]", "DATA[5]", "DATA[6]", "DATA[7]", "Flag"]

        df_car_hacking = df_car_hacking.toDF(*columnas)

        # Identifica tipos de datos
        df_car_hacking = df_car_hacking.withColumn("Timestamp", col("Timestamp").cast("double"))
        df_car_hacking = df_car_hacking.withColumn("DLC", col("DLC").cast("int"))

    elif archivo.endswith(".txt"):
        # Lee el archivo de texto
        df_text = spark.read.text(archivo)

        # Extrae los campos del archivo de texto
        df_car_hacking = df_text.select(
            regexp_extract("value", r"Timestamp: ([0-9.]+)", 1).alias("Timestamp"),
            regexp_extract("value", r"ID: ([0-9a-fA-F]+)", 1).alias("CAN ID"),
            regexp_extract("value", r"DLC: (\d+)", 1).alias("DLC"),
            regexp_extract("value", r"DLC: \d+ ([0-9A-Fa-f]{2})", 1).alias("DATA[0]"),
            regexp_extract("value", r"DLC: \d+ ([0-9A-Fa-f]{2}) {1}([0-9A-Fa-f]{2})", 1).alias("DATA[1]"),
            regexp_extract("value", r"DLC: \d+ ([0-9A-Fa-f]{2}) {2}([0-9A-Fa-f]{2})", 1).alias("DATA[2]"),
            regexp_extract("value", r"DLC: \d+ ([0-9A-Fa-f]{2}) {3}([0-9A-Fa-f]{2})", 1).alias("DATA[3]"),
            regexp_extract("value", r"DLC: \d+ ([0-9A-Fa-f]{2}) {4}([0-9A-Fa-f]{2})", 1).alias("DATA[4]"),
            regexp_extract("value", r"DLC: \d+ ([0-9A-Fa-f]{2}) {5}([0-9A-Fa-f]{2})", 1).alias("DATA[5]"),
            regexp_extract("value", r"DLC: \d+ ([0-9A-Fa-f]{2}) {6}([0-9A-Fa-f]{2})", 1).alias("DATA[6]"),
            regexp_extract("value", r"DLC: \d+ ([0-9A-Fa-f]{2}) {7}([0-9A-Fa-f]{2})", 1).alias("DATA[7]"),
        )

        # Identifica tipos de datos
        df_car_hacking = df_car_hacking.withColumn("Timestamp", col("Timestamp").cast("double"))
        df_car_hacking = df_car_hacking.withColumn("DLC", col("DLC").cast("int"))
        df_car_hacking = df_car_hacking.withColumn("Flag", lit("R"))

```

Fig 4. Función de integración y procesamiento de archivos CSV, TXT. Fuente: Autor.

3. Integración de archivos y muestreo de registros:

Una vez que los archivos están procesados individualmente, se crea una lista de DataFrames para que posteriormente se integren en un único dataset mediante la función unionByName.

```

dataframes = []

for archivo, tipo in archivo_tipo.items():
    df_tmp = carga_integracion(archivo, tipo)
    dataframes.append(df_tmp)

# Unir todos en un solo DataFrame
df_unificado = dataframes[0]
for df in dataframes[1:]:
    df_unificado = df_unificado.unionByName(df)

```

Fig 5. Integración de los archivos procesados. Fuente: Autor.

4.1.2 Transformación

Se realiza la transformación de dos campos con el objetivo de facilitar el análisis exploratorio y el posterior modelado de datos, en este caso se hace la conversión de aquellos campos con valores

en formato hexadecimal a formato entero, los cuales son CAN ID (identificador del bus CAN) y DATA[0] a DATA[7] (Valores de los datos del mensaje CAN dentro de un formato byte de 8 bits).

Para el desarrollo, fue necesario definir una función de usuario (UDF) la cual permite interpretar valores hexadecimales y convertirlos a enteros utilizando la base 16, de esta transformación resultaron nuevas columnas llamadas CAN_ID_dec y DATA_0_int a DATA_7_int.

```
# UDF para convertir de hexadecimal a entero
def hex_to_int(hex_str):
    try:
        return int(hex_str, 16)
    except:
        return None

hex_to_int_udf = udf(hex_to_int, IntegerType())

# Conversión de CAN ID
df_car_hacking = df_car_hacking.withColumn("CAN_ID_hex", regexp_replace(col("CAN_ID"), "0x", ""))
df_car_hacking = df_car_hacking.withColumn("CAN_ID_dec", hex_to_int_udf(col("CAN_ID_hex")))
df_car_hacking = df_car_hacking.drop("CAN_ID_hex")

# Conversión de columnas DATA[i] de hex a int
for i in range(8):
    df_car_hacking = df_car_hacking.withColumn(f"DATA_{i}_int", hex_to_int_udf(col(f"DATA[{i}]")))
```

Fig 6. Transformación de campos con tipo de dato hexadecimal a entero. Fuente: Autor.

4.1.3 Etiquetado

Para el proceso del modelado el etiquetado es una fase importante, es por eso que se implementó dentro de la función carga_integracion un sistema de etiquetado multiclase donde se crea una nueva columna denominada "Tipo_ataque", la cual se basa en asignar una categoría a cada registro según el tipo de archivo (pedido por parámetro) del que proviene. Ejemplo: Normal_Trafic, Spoofing_RPM_Attack, Spoofing_Gear_Attack, Fuzzing_Attack y DoS_Attack. Por otra parte, se realizó una etiqueta binaria representada en la columna Ataque, la cual refiere a si el registro corresponde a algún tipo de ataque toma el valor de true si corresponde a un tráfico normal o false de lo contrario.

```
# Etiquetas multiclase y binaria
df_car_hacking = df_car_hacking.withColumn("Tipo_ataque", lit(tipo_ataque))
df_car_hacking = df_car_hacking.withColumn("Ataque", when(col("Tipo_ataque") == "Normal_Trafic", False).otherwise(True))
```

Fig 7. Proceso de etiquetado multiclase y binario. Fuente: Autor.

4.1.4 Exportación del Dataset Unificado

Finalizando este proceso de carga, procesamiento e integración, el DataFrame resultante se exporta como un único archivo csv, el dataset unificado cuenta con un total de siete atributos, a diferencia del dataset original. A continuación se aprecia las diferencias entre ambos:

Atributos del Dataset Actual	Tipo de dato Dataset Actual	Atributos del Dataset Original	Tipo de dato Dataset Original
Timestamp	Double	Timestamp	Double
CAN_ID_dec	Integer	CAN ID	String (Hexadecimal)
DCL	Integer	DCL	Integer
DATA_0_int a DATA_7_int	Integer	DATA[0] a DATA[7]	String (Hexadecimal)
Flag	String	Flag	String
Tipo_ataque	String		
Ataque	Boolean		

Tabla 3. Diferencias de los atributos y los tipos de datos entre datasets. Fuente: Autor.

Para consultar el conjunto de datos resultante, diríjase al siguiente enlace [Dataset - Estructurado](#), éste proporciona el acceso al archivo csv con los datos ya estructurados.

4.2 Análisis de Datos

Este apartado se desarrolló con el objetivo de identificar patrones de comportamiento normales como anormales en las señales vehiculares, donde se implementa un EDA para estudiar la estructura del dataset, la distribución de variables y la representación visual de las mismas con el fin de caracterizar los ataques.

Mediante visualizaciones estadísticas y descriptivas se facilita la comprensión de la naturaleza de los datos, permitiendo detectar comportamientos referentes a las variables pertenecientes al dataset.

4.2.1 Análisis Exploratorio de Datos (EDA)

Por medio de este análisis se busca obtener una comprensión profunda y detallada del conjunto de datos antes de aplicar modelos de machine learning, esto será posible mediante la detección de patrones, relaciones, tendencias, valores atípicos y posibles inconsistencias en los datos ya sean errores, valores faltantes, entre otros.

Para este proceso, se implementan diversas técnicas para examinar el conjunto de datos desde tres enfoques complementarios los cuales serán explicados a lo largo de este apartado.

4.2.1.1 Estructura del Conjunto de Datos

Se analiza la estructura del dataset para comprender los tipos de datos con los que se trabajará y se evaluará su calidad, este proceso es fundamental debido a que se detectan posibles inconsistencias o problemas que podrían afectar la interpretación de resultados o el desarrollo de modelos, es por eso que a través de esta revisión se busca garantizar la integridad del conjunto de datos y determinar si se requiere realizar transformaciones previas antes de avanzar con el análisis y modelado.

Inicialmente, se verificó la estructura del esquema para que todos los campos correspondan al tipo de dato que manejan para así evitar posibles errores en el resto del análisis. El esquema resultante muestra un diverso conjunto de tipos de datos que cuentan con: 1 columna tipo double, 10 columnas tipo entero, 2 columnas tipo string, y 1 columna tipo boolean, donde cada columna puede tener valores nulos.

```
df_car_hacking.printSchema()

root
|-- Timestamp: double (nullable = true)
|-- DLC: integer (nullable = true)
|-- Flag: string (nullable = true)
|-- Tipo_ataque: string (nullable = true)
|-- Ataque: boolean (nullable = true)
|-- CAN_ID_dec: integer (nullable = true)
|-- DATA_0_int: integer (nullable = true)
|-- DATA_1_int: integer (nullable = true)
|-- DATA_2_int: integer (nullable = true)
|-- DATA_3_int: integer (nullable = true)
|-- DATA_4_int: integer (nullable = true)
|-- DATA_5_int: integer (nullable = true)
|-- DATA_6_int: integer (nullable = true)
|-- DATA_7_int: integer (nullable = true)
```

Fig 8. Esquema del conjunto de datos. Fuente: Autor.

La identificación de valores nulos constituye un proceso fundamental para evaluar la calidad del conjunto de datos y determinar las estrategias más adecuadas para el tratamiento de columnas con un alto porcentaje de datos faltantes. En la siguiente tabla, se observa que algunas variables, como Flag, presenta un porcentaje de 1,14 de valores nulos, mientras que otras como DATA_0_int a DATA_7_int superan el 5% e incluso marcan un 6.77% de valores faltantes.

Dado que estas columnas contienen información relevante para el análisis posterior, optar por eliminarlas no daría una solución, si no que implicaría una pérdida significativa de datos, es por eso que, se podría considerar la imputación como una alternativa viable para preservar la integridad del conjunto.

	Atributo	Cant. Valores Nulos	Porcentaje (%)
0	DLC	1	0.00
1	CAN_ID_dec	1	0.00
2	Flag	200665	1.14
3	DATA_0_int	988872	5.63
4	DATA_1_int	988872	5.63
5	DATA_2_int	1136083	6.47
6	DATA_3_int	1136083	6.47
7	DATA_4_int	1136083	6.47
8	DATA_5_int	1189534	6.77
9	DATA_6_int	1189537	6.77
10	DATA_7_int	1189537	6.77

Tabla 4. Cantidad de valores nulos y porcentaje en la columna encontrada. Fuente: Autor.

La identificación de la cantidad de valores únicos presentes en cada columna del conjunto permite comprender la variabilidad de las variables y evaluar la relevancia dentro del análisis, siendo así que aquellas columnas que presentan una diversidad muy limitada de valores no aportan mayor información significativa debido a que no permiten distinguir patrones o relaciones entre variables.

Por otro lado, no siempre se deben excluir o eliminar aquellos campos con baja cardinalidad, en este caso puntualmente columnas como Flag, Ataque y Tipo_Ataque, a pesar de tener 2 o 5 valores únicos su relevancia se centra en su función como indicadores clave para la detección de anomalías en entornos de vehículos conectados siendo esenciales para tareas de clasificación, detección de señales intrusas o validación de comportamiento normal.

Conteo de valores únicos en el dataframe:													
Timestamp	DLC	Flag	Tipo_ataque	Ataque	CAN_ID_dec	DATA_0_int	DATA_1_int	DATA_2_int	DATA_3_int	DATA_4_int	DATA_5_int	DATA_6_int	DATA_7_int
14885143	4	2	5	2	2048	256	256	256	256	256	256	256	256

Tabla 5. Cantidad de valores únicos. Fuente: Autor.

4.2.1.2 Distribución de Variables

Se analizan las características estadísticas de las variables presentes en el conjunto de datos, con el fin de detectar posibles sesgos, desbalances o niveles elevados de variabilidad. En la tabla presentada, se observa que los campos DATA_0_int a DATA_7_int representan diferentes segmentos en el protocolo CAN, presentando un rango entre 0 y 255, siendo coherente con el formato estándar de los datos de 1 byte (8 bits), lo cual significa que estos datos han sido capturados de la manera esperada y se encuentran dentro de los límites para este tipo de comunicación vehicular.

Además, campos como DLC y CAN_ID_dec muestran valores mínimos y máximos razonables, lo que refiere a que los registros no presentan valores fuera de rango, lo cual explica que no se encontraron anomalías sintácticas o errores de codificación al extraer la información. Por otro

lado, se observa que tanto Flag como Tipo_ataque que son valores categóricos y como no se les aplican como tal las estadísticas numéricas tradicionales cumplen un papel clave en la identificación de ataques o comportamientos anómalos en la red vehicular.

summary	Timestamp	DLC	Flag	Tipo_ataque	CAN_ID_dec	DATA_0_int	DATA_1_int	DATA_2_int	DATA_3_int	DATA_4_int	DATA_5_int	DATA_6_int	DATA_7_int
count	17558347	17558346	17557682	17558347	17558346	16569475	16422264	16422264	16422264	16368813	16368818	16368818	16368818
mean	1.4782486358909137E9	7.928454707521995	NULL	NULL	704.8344257483822	58.0421416490263	45.52503317093632	40.39390123967875	78.47419460556718	51.74480893742787	61.89311485200545	26.326757717879387	54.20127712399374
stddev	213309.11005279864	0.6089453842002505	NULL	NULL	396.1405571036111	89.85980802170869	54.4582137671578	58.51307775299792	102.333442353933	73.34574177202056	77.33790197477545	57.41984764874757	80.20979815346993
min	1.478191830045114E9	2	R	DoS_Attack	0	0	0	0	0	0	0	0	0
max	1.479121941287013E9	8	T	Spoofing_RPH_Attack	2047	255	255	255	255	255	255	255	255

Tabla 6. Distribución estadística. Fuente: Autor.

Se analiza el balance del dataset basado en la columna Ataque, debido a que se identifica como la posible variable objetivo para el problema de detección de anomalías (modelo binario) en vehículos conectados, en comparación a otras variables que pueden ser más detalladas y describir características o estados del sistema, la distribución de la columna Ataque impacta directamente en el desempeño del modelo predictivo enfocado en la presencia o ausencia de señales maliciosas.

Como se aprecia en la tabla, el dataset presenta un desbalance significativo entre las clases lo cual puede afectar negativamente el rendimiento del modelo, es por eso que, antes o durante la fase de modelado, será necesario aplicar técnicas de balanceo de datos. Este tipo de análisis exploratorios ayuda a identificar este tipo de desproporciones para posteriormente mitigar posibles sesgos y mejorar la capacidad predictiva del modelo.

Ataque	count	Porcentaje
true	16569475	94.3680803210006
false	988872	5.631919678999395

Tabla 7. Balance del dataset. Fuente: Autor.

4.2.1.3 Visualización

Para el análisis exploratorio, la visualización es una herramienta esencial para representar gráficamente el comportamiento de variables y sus relaciones, teniendo en cuenta la identificación de patrones, tendencias, valores atípicos y correlaciones que podrían no ser evidentes a través del análisis estadístico.

En el análisis realizado sobre la frecuencia de señales (ataques o normales), se evidencia una marcada proporción de registros correspondientes a comportamientos anómalos, específicamente los ataques tipo Spoofing en señales de RPM y cambios de marcha que alcanzan aproximadamente los 4,7 y 4,5 millones de registros respectivamente, a comparación con el tráfico normal que presenta una frecuencia notablemente menor. Para los ataques como Fuzzing y DoS no se encuentran en la mitad de registros, ya que también poseen una frecuencia ligeramente menor a los dos ataques más frecuentes.

Este comportamiento sugiere un importante desbalance en el dataset hacia señales de naturaleza maliciosa, lo cual beneficia directamente a la futura efectividad de un modelo de detección de anomalías, ya que este enfoque ayudará a anticipar escenarios reales en los que los sistemas vehiculares puedan estar expuestos a ataques de este tipo.

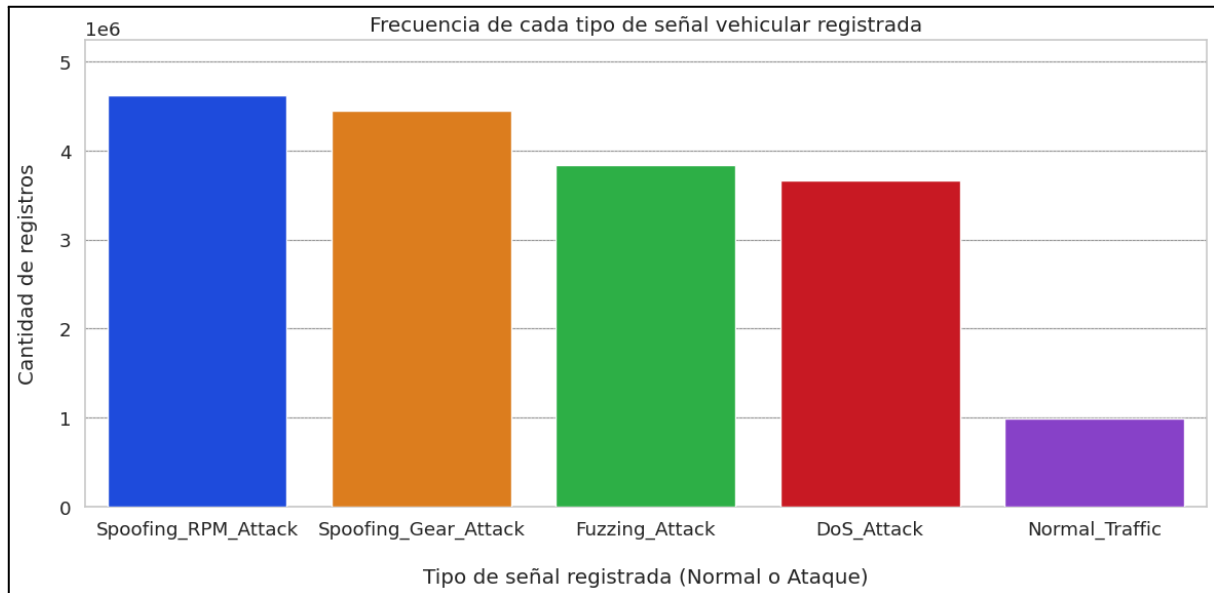


Fig 9. Frecuencia de registros por tipo de señales normales o ataques. Fuente: Autor.

El gráfico de frecuencia temporal registrada a lo largo del tiempo, permite observar cómo varía la cantidad de mensajes registrados por cada categoría en distintos intervalos temporales, donde se evidencia un comportamiento lineal creciente referente a Normal_Traffic, la cual progresivamente se intensifica a lo largo del tiempo, a diferencia de los registros de los ataques. Para Spoofing_Gear y Spoofing_RPM, registran una frecuencia inicialmente alta que va decreciendo de forma constante a medida en la que transcurre el tiempo lo que indica que los ataques fueron más frecuentes en las etapas iniciales de monitoreo y posteriormente el tráfico normal aumentó en las etapas posteriores.

Por otra parte, en los primeros segundos se evidencia una actividad intensa de todos los tipos de señal, lo cual indica que las pruebas se realizaron en un entorno altamente intervenido o manipulado y a pesar de que los ataques como DoS y Fuzzing no se aprecian de una manera más marcada, se analiza que presentan una distribución temporal segmentada lo cual puede estar vinculada a momentos de ataque específicos. Este tipo de análisis permite detectar y prevenir en ventanas de tiempo específicas, la posible aparición de un comportamiento anómalo donde la actividad maliciosa se intensifica.

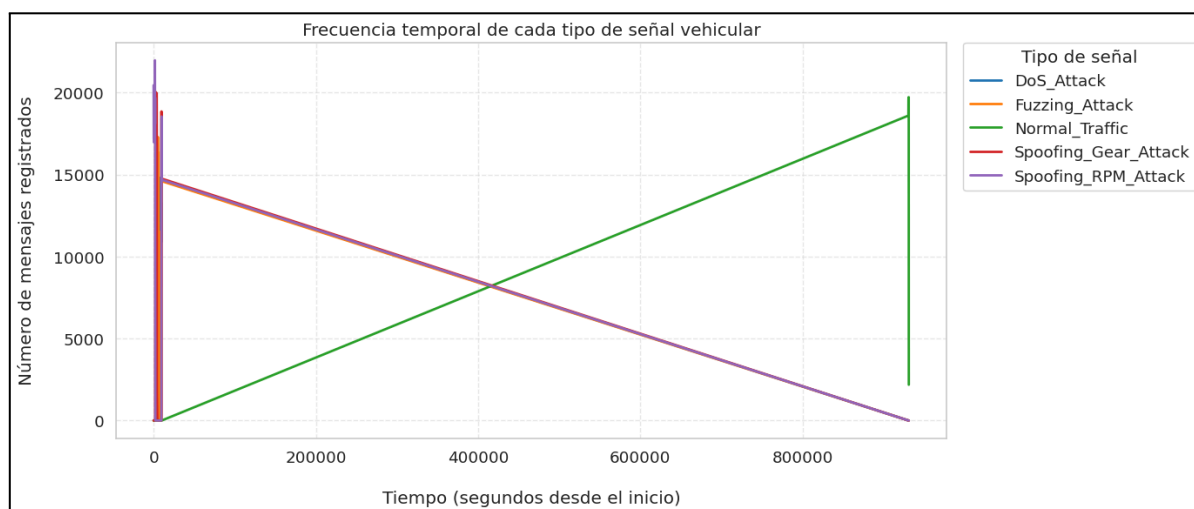


Fig 10. Distribución temporal de señales vehiculares normales y de ataque. Fuente: Autor.

En el siguiente gráfico. se presenta la frecuencia de mensajes por cada identificador CAN, donde algunos se concentran en una mayor cantidad de mensajes superando los 200.000 registros, en algunos casos como los indicadores cercanos a 400 y 1000 indican la posible existencia de identificadores particularmente sensibles o frecuentemente actualizados tanto en tráfico normal como en ataques, además también se observa que todos los tipos de señal están presentes en múltiples rangos de identificadores variando su intensidad.

Por lo tanto, se observa que la distribución no es uniforme ni exclusiva, ya que muchos indicadores son compartidos entre el tráfico normal y los ataques, lo cual resulta limitante la identificación de anomalías basada únicamente en el campo identificador CAN.

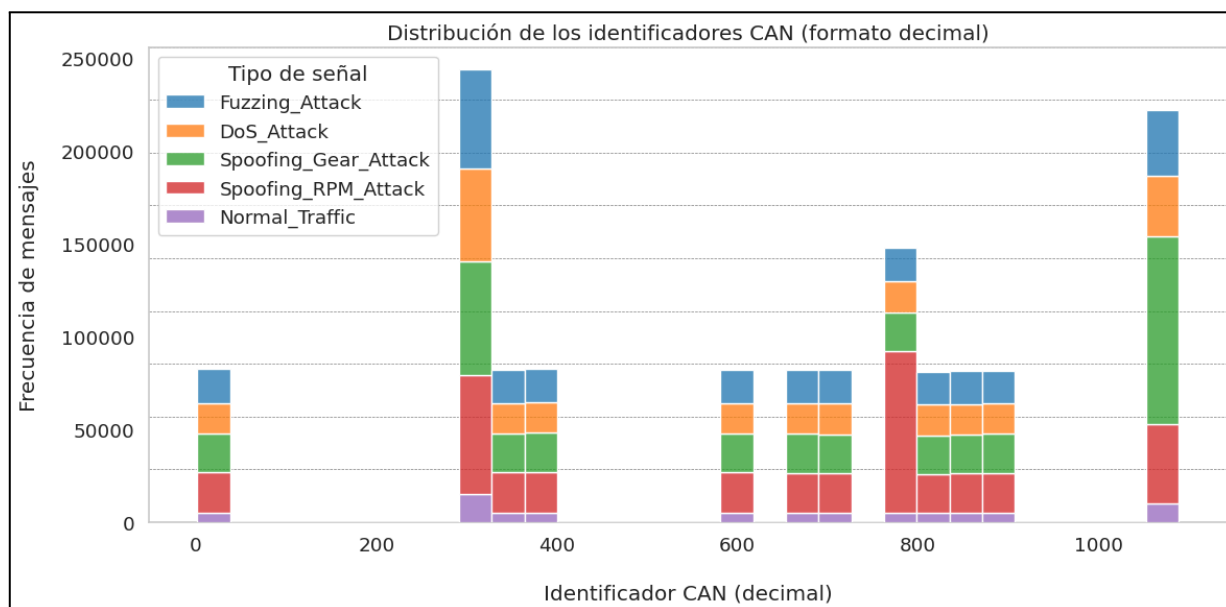


Fig 11. Frecuencia de mensajes según identificador CAN y tipo de señal vehicular. Fuente: Autor.

En el siguiente gráfico, se observa que la gran mayoría de los tipos de señal presentan una longitud de 8 bytes, que corresponde al tamaño máximo permitido por el protocolo CAN estándar, según el gráfico los ataques Spoofing_RPM y Spoofing_Gear son los que más contribuyen al

volumen de mensajes, seguidos por los ataques DoS y Fuzzing, pero la señal de tráfico normal representa una proporción considerablemente baja dentro de este tamaño.

Por otro lado, en los tamaños inferiores como 2 y 5 bytes se registra una actividad secundaria que se distribuye de manera dispersa entre los distintos tipos de señal sin un patrón dominante. Esta baja representación sugiere que tanto en condiciones normales como durante los ataques, los mensajes más pequeños no son utilizados con una frecuencia significativa.

Este patrón indica una preferencia operativa o estratégica por el uso de mensajes de longitud máxima, posiblemente con el fin de transmitir mayor volumen de datos en un solo paquete o para dificultar la detección de anomalías al camuflar mensajes maliciosos entre los normales, de esta manera el campo DLC es un componente clave para poder identificar desviaciones sutiles en el comportamiento de la red vehicular.

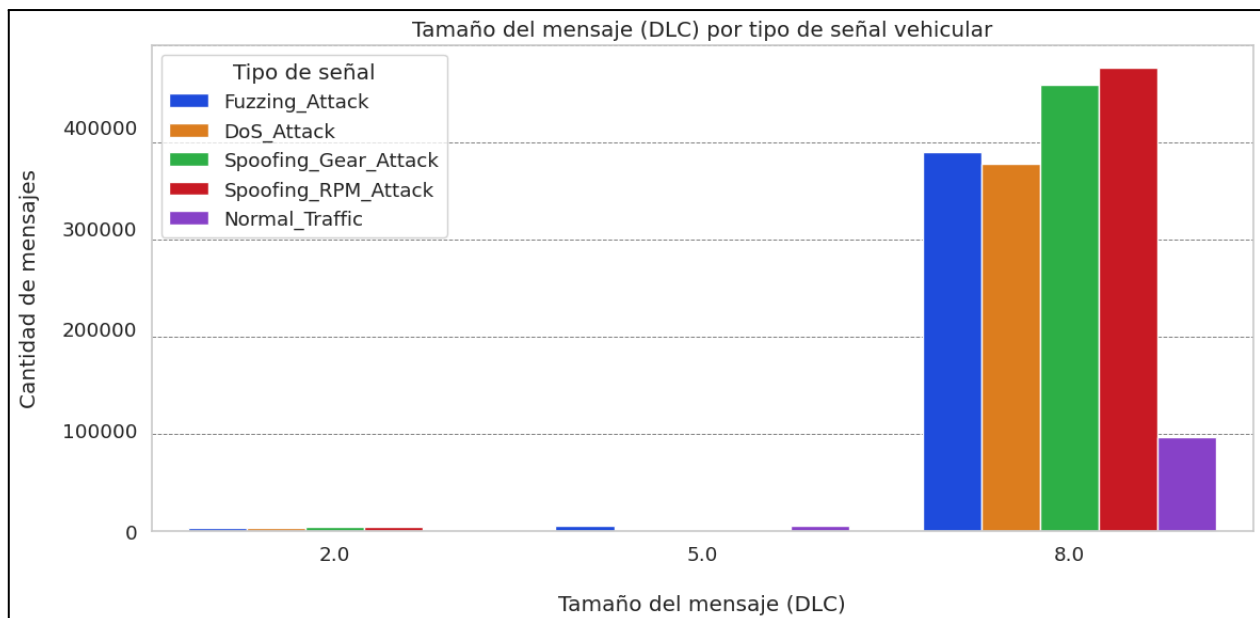


Fig 12. Distribución del tamaño de los mensajes (DLC) por tipo de señal vehicular. Fuente: Autor.

En el gráfico se aprecia que los valores promedio varían notablemente según el tipo de señal y la posición del byte, donde el byte del mensaje DATA_3_int se destaca por tener valores muy elevados en los ataques de tipo Spoofing_Gear y Spoofing_RPM superando los 80 bytes, mientras que el tráfico normal mantiene un nivel moderado cercano a 70 bytes, otro caso relevante es el byte del mensaje DATA_5_int, donde el ataque DoS_Attack muestra un valor promedio considerablemente más alto en comparación con otros tipos de señal. Los bytes del mensaje DATA_0_int, DATA_1_int y DATA_2_int presentan valores más dispersos, pero sin diferencias tan marcadas como en los casos anteriores.

De acuerdo a lo mencionado, la identificación de bytes con valores atípicos o consistentemente alterados en ataques facilita la discriminación entre señales normales y anómalas incrementando

la precisión de los sistemas de seguridad vehicular, esta distribución diferencial sugiere que los ataques modifican patrones específicos dentro de los mensajes.

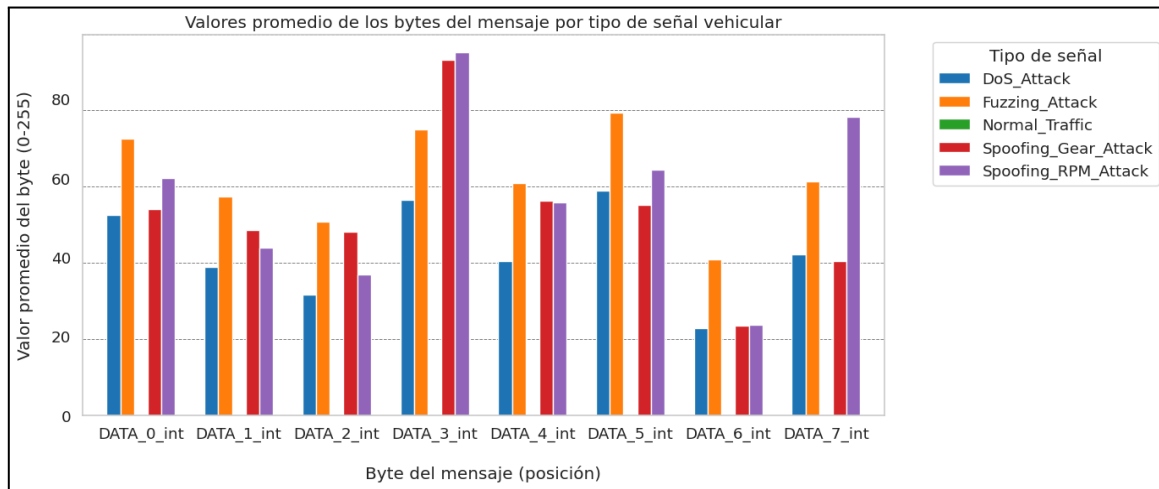


Fig 13. Valores promedio de los bytes en mensajes CAN según el tipo de señal vehicular. Fuente: Autor.

Como se puede apreciar en el gráfico, para todos los tipos de señal la mayoría de los mensajes corresponden a la categoría de mensajes regulares (R) con valores que fluctúan aproximadamente entre 3 y 4 millones respecto a Spoofing_RPM y Spoofing_Gear y ligeramente menor para Fuzzing y DoS, demostrando que predomina para esta categoría, los ataques. En el caso de la categoría de mensajes manipulados (T) son considerablemente menores con un patrón similar en todos los tipos de ataques ausentando en este caso al tráfico normal.

Este balance entre mensajes legítimos y manipulados indica que aunque la mayor parte del tráfico vehicular se mantiene en un estado regular, la proporción de mensajes manipulados es mayormente significativa como para representar una amenaza real y constante. La discriminación entre mensajes regulares y manipulados es crucial para desarrollar estrategias que permitan no solo detectar la presencia de ataques, sino también identificar su modo de operación y evolución en el tiempo.

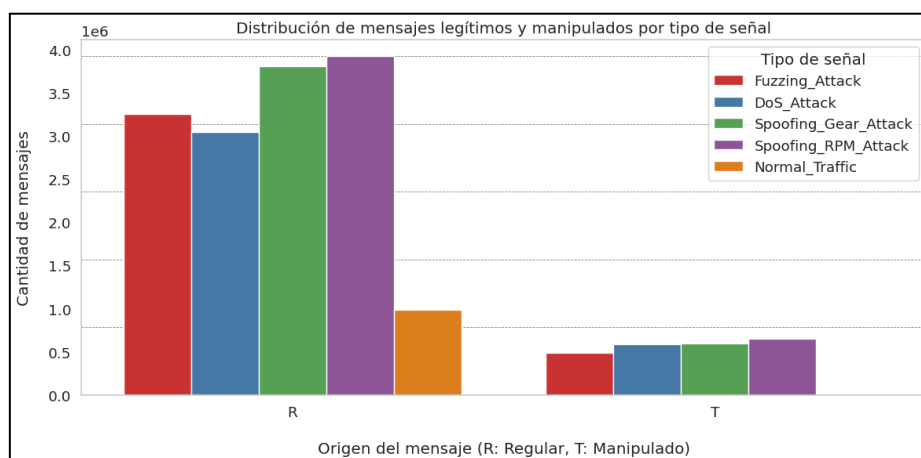


Fig 14. Distribución de mensajes legítimos y manipulados según tipo de señal vehicular. Fuente: Autor.

En el siguiente gráfico, se aprecia que los identificadores para los distintos tipos de ataque y el tráfico normal comparten rangos similares, con medianas situadas aproximadamente entre 600 y 800 registros, los ataques Fuzzing y Spoofing_Gear presentan la mayor dispersión que se extiende desde valores cercanos a cero hasta casi 2.000, indicando que estos ataques abarcan una amplia gama de identificadores CAN. Por otro lado, para el ataque Spoofing_RPM muestra también un rango amplio, aunque con algunos valores atípicos que superan el límite superior habitual, lo que puede indicar patrones inusuales o picos en determinados identificadores, además, el ataque DoS y el tráfico normal tienen una dispersión algo menor concentrándose en rangos intermedios.

La superposición en la distribución de identificadores entre tráfico normal y ataques resalta la dificultad de diferenciar estos estados sólo con base en el identificador CAN, no obstante, la amplitud y presencia de valores extremos en algunos ataques sugiere que el análisis estadístico detallado y la identificación de valores atípicos pueden ser herramientas útiles para detectar anomalías.

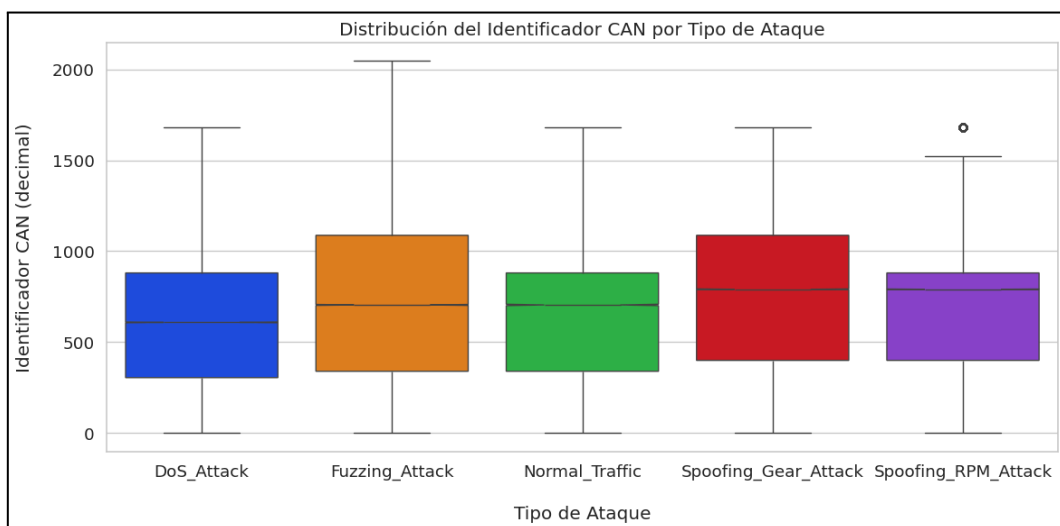


Fig 15. Distribución del identificador CAN por tipo de señal vehicular. Fuente: Autor.

En la siguiente matriz, se puede destacar que la variable DATA_5_int tiene una correlación moderada y positiva con DATA_3_int con un valor de 0.41 y con DATA_4_int con un valor de 0.27, esto sugiere que los anteriores bytes tienden a variar de manera conjunta, así mismo, DATA_2_int muestra correlaciones positivas moderadas con DATA_1_int con un valor de 0.26 y DATA_4_int con un valor de 0.35, lo que podría indicar patrones de dependencia entre ciertos bytes del mensaje CAN. La variable CAN_ID_dec muestra correlaciones moderadas positivas con algunos bytes, como DATA_4_int con un valor de 0.30 y DATA_3_int con un valor de 0.20, lo cual puede reflejar la influencia del identificador en el contenido del mensaje, las demás variables como Timestamp y DLC presentan correlaciones bajas o cercanas a cero con la mayoría de las variables lo cual sugiere que su relación con el contenido del mensaje es débil o nula.

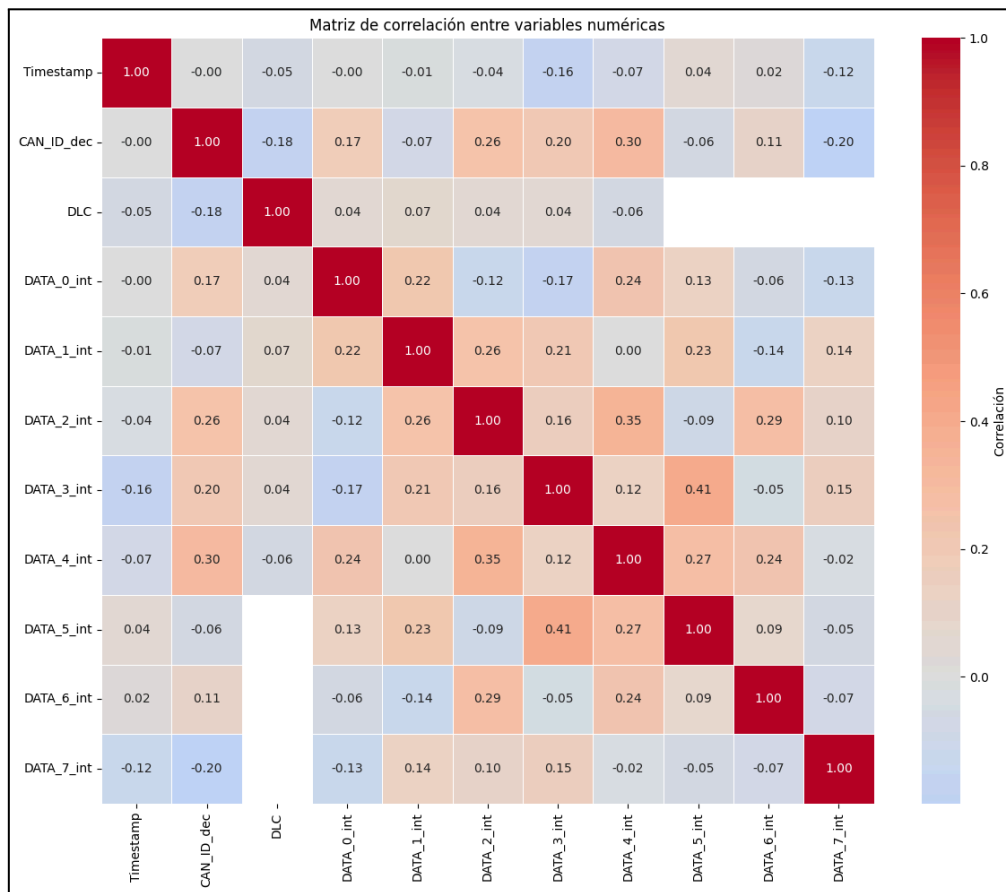


Fig 16. Matriz de correlación entre variables numéricas. Fuente: Autor.

En el siguiente gráfico, se evidencia que en algunos indicadores como los ubicados alrededor de 760 y 1070, presentan frecuencias significativamente superiores al promedio superando incluso la desviación estándar, lo cual indica que ciertos indicadores son mucho más utilizados o transmitidos en la red vehicular relacionados posiblemente a funciones críticas o estados importantes del vehículo. Por otro lado, hay identificadores que muestran frecuencias más homogéneas y cercanas a la media lo cual indica una distribución más equilibrada para esos casos, al identificar estos CAN ID's con alta frecuencia ayuda al desarrollo de estrategias de monitoreo y detección, ya que pueden resultar en puntos vulnerables donde los ataques podrían concentrarse.

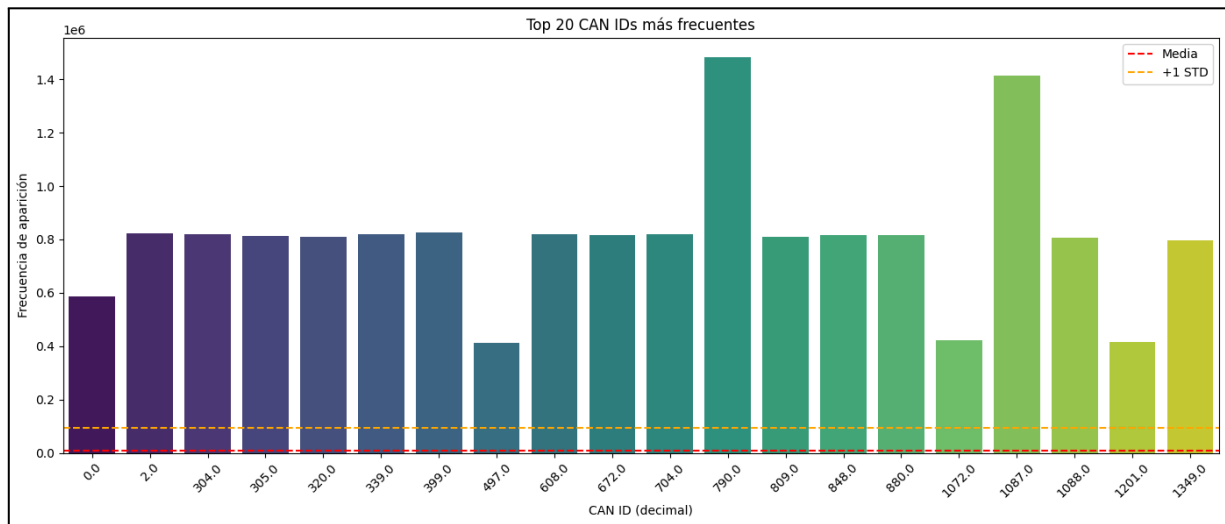


Fig 17. Frecuencia de los 20 identificadores CAN más comunes en la red vehicular. Fuente: Autor.

La entropía presentada a continuación, representa la medida de aleatoriedad o incertidumbre en los datos de la red vehicular, siendo un indicador clave para detectar comportamientos anómalos, donde se observa que el tráfico normal presenta una entropía muy baja o nula lo que sugiere una estructura altamente predecible y repetitiva en los mensajes con un sistema vehicular operando sin alteraciones.

Por otro lado, todos los ataques muestran distribuciones de entropía significativamente más altas, con valores que superan ampliamente los 2 bits y en algunos casos valores cercanos a 3.5 bits, los ataques presentan formas complejas y multimodales, que refiere que los patrones de aleatoriedad varían según el tipo de intrusión, en este caso ataques como Fuzzing y Spoofing_RPM tienen a mostrar una mayor dispersión siendo un comportamiento menos predecible y posiblemente más ruidoso (irregular), por otro lado, el ataque DoS a pesar de que genera una alta entropía, presenta picos en valores cercanos a cero que puede implicar momentos de comportamiento repetitivo o bloqueante.

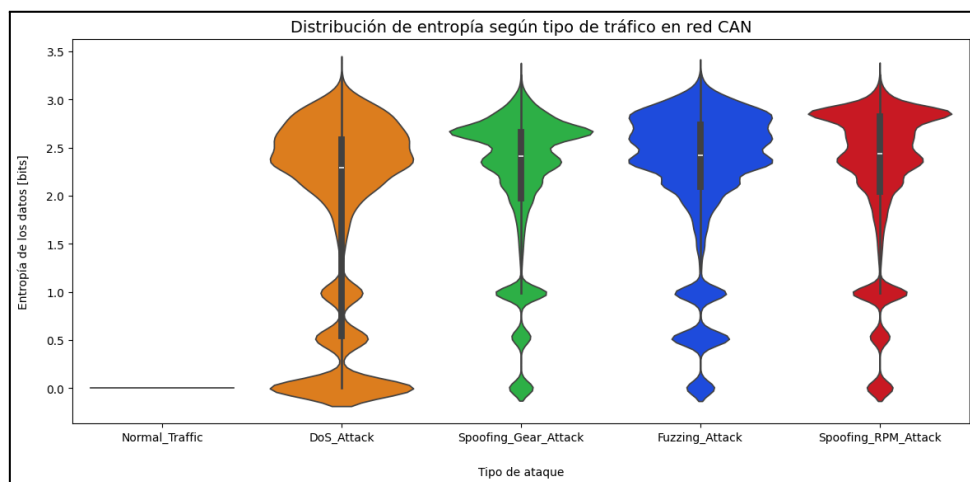


Fig 18. Distribución de entropía en la red CAN según el tipo de señal vehicular. Fuente: Autor.

4.3 Modelado

En este apartado se describe el proceso llevado a cabo para el desarrollo y entrenamiento de modelos orientados a la detección y clasificación de ciberataques en vehículos conectados, específicamente sobre redes CAN. Para ello, se ha seguido un enfoque integral que incluye múltiples técnicas de preprocesamiento de datos, ingeniería de características, balanceo de clases, normalización y la implementación de diferentes tipos de modelos de aprendizaje automático, tanto supervisados como no supervisados.

El enfoque propuesto se compone de la siguiente manera:

- Modelo de clasificación binaria (supervisado), se utilizará como primer filtro dentro del sistema, ya que es el encargado de determinar si una muestra corresponde a una señal normal o a un tipo de ataque.
- Modelo de clasificación multiclase (supervisado), se aplica únicamente sobre aquellas muestras que han sido identificadas como ataques en el modelo binario, con el objetivo de determinar el tipo específico de ataque.
- Modelo Isolation Forest (no supervisado), el cual es utilizado como método alternativo de detección basado en el aprendizaje de patrones normales y la identificación de anomalías.

Adicionalmente, durante el desarrollo del proyecto se agregaron nuevos modelos que no estaban contemplados en la propuesta inicial, con el objetivo de ampliar las capacidades del sistema y mejorar el rendimiento

- Pipeline Híbrido (combinación de los modelos binario y multiclase), se utiliza para integrar las capacidades del modelo binario y multiclase, permitiendo así una clasificación más robusta y la incorporación de una categoría adicional para señales sospechosas que no puedan clasificarse de una manera específica, para este caso se las denomina como “desconocido”.
- Modelo Autoencoder Denso (no supervisado), se implementa como alternativa eficiente a modelos secuenciales, el cual está orientado a la detección de anomalías a partir del error de reconstrucción.

Los siguientes apartados detallan cada una de las etapas involucradas en este pipeline que indica el proceso desde la preparación del conjunto de datos, hasta el entrenamiento, evaluación y comparación de los modelos desarrollados.

4.3.1 Preparación y Limpieza de datos

De acuerdo a los hallazgos encontrados y presentados en el EDA, se llevaron a cabo las siguientes acciones:

- **Tratamiento de valores nulos:** Se identificaron 10 columnas con valores nulos, de los cuales para no perder tantos registros se optó por implementar una técnica de imputación, la cual consiste en reemplazar los valores nulos numéricos por la media de cada atributo correspondiente y a su vez se imputaron valores categóricos como la columna Flag, que se imputó por el valor más frecuente en este caso la categoría "R".

```
# Limpieza e imputación
for i in range(8):
    col_name = f"DATA_{i}_int"
    mean_val = df_car_hacking.select(mean(col(col_name))).first()[0]
    if mean_val is not None:
        df_car_hacking = df_car_hacking.fillna({col_name: int(mean_val)})

df_car_hacking = df_car_hacking.fillna({'Flag': 'R'})
df_car_hacking = df_car_hacking.filter(col("CAN_ID_dec").isNotNull() & col("DLC").isNotNull())

df_car_hacking.select([count(when(col(c).isNull(), c)).alias(c) for c in df_car_hacking.columns]).show()
```

Timestamp	DLC	Flag	Tipo_ataque	Ataque	CAN_ID_dec	DATA_0_int	DATA_1_int	DATA_2_int	DATA_3_int	DATA_4_int	DATA_5_int	DATA_6_int	DATA_7_int	Entropia
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Fig 19. Limpieza e imputación de valores nulos. Fuente: Autor.

- **Conversión del dataframe:** Para el desarrollo de los modelos fue necesario convertir el conjunto de datos original al formato Pandas DataFrame debido a su flexibilidad para el análisis y manipulación de los datos, aunque no es el más óptimo al momento de manejar grandes volúmenes de información, por ese motivo se optó por reducir la muestra al 20% del total para así optimizar el rendimiento de procesamiento.

```
# Reducir muestra (para rendimiento)
df_sampled = df_car_hacking.sample(fraction=0.2, seed=42)
df_unificado = df_sampled.toPandas()
```

Fig 20. Reducción del dataframe. Fuente: Autor

- **Análisis de entropía:** Previamente se calculó la variable entropía durante el análisis exploratorio y debido a su potencial para discriminar entre eventos maliciosos y normales, de esta manera se optó por incorporar esta variable como una característica adicional en el conjunto de entrenamiento. El siguiente enlace [Dataset - Modelado](#), proporciona el acceso al archivo csv con los datos utilizados para el modelo.

```

|-- Timestamp: double (nullable = true)
|-- DLC: integer (nullable = true)
|-- Flag: string (nullable = true)
|-- Tipo_ataque: string (nullable = true)
|-- Ataque: boolean (nullable = true)
|-- CAN_ID_dec: integer (nullable = true)
|-- DATA_0_int: integer (nullable = true)
|-- DATA_1_int: integer (nullable = true)
|-- DATA_2_int: integer (nullable = true)
|-- DATA_3_int: integer (nullable = true)
|-- DATA_4_int: integer (nullable = true)
|-- DATA_5_int: integer (nullable = true)
|-- DATA_6_int: integer (nullable = true)
|-- DATA_7_int: integer (nullable = true)
|-- Entropia: double (nullable = true)

```

Fig 21. Esquema del conjunto de datos para el modelado. Fuente:Autor.

4.3.2 Desarrollo del Modelo

Se desarrollaron tres versiones iterativas del entorno de entrenamiento y evaluación, las cuales se diseñaron y ejecutaron en fases sucesivas con el objetivo de superar limitaciones detectadas en la implementación inicial. En este caso, cada versión representó una mejora técnica y significativa en relación a precisión, tiempo de entrenamiento y manejo de balanceo de clases.

Los ajustes realizados están relacionados mayormente con el modelo supervisado multiclase, donde se incluyeron nuevas variables y modificación de las estrategias de balanceo de datos, estas mejoras se reflejaron en un desempeño superior con respecto a versiones anteriores, evidenciando un proceso progresivo fundamentado en los resultados obtenidos en cada fase.

Dado al volumen de los datos, se implementó Apache Spark para el procesamiento inicial y limpieza, posteriormente los datos fueron convertidos a un DataFrame de Pandas para realizar el modelado y entrenamiento con librerías como scikit-learn y XGBoost por mayor flexibilidad y compatibilidad. Inicialmente se contempló el uso de Spark ML, pero esta opción fue descartada debido a incompatibilidades con ciertos modelos no supervisados, como Isolation Forest, que presentó errores relacionados con versiones.

Cabe aclarar que las siguientes técnicas fueron aplicadas de manera común a todos los modelos supervisados desarrollados (clasificación binaria, multiclase y pipeline híbrido) con adaptaciones específicas en algunos casos.

- a. Ingeniería de características:** Este proceso permite crear, transformar, seleccionar y preparar las variables o atributos que se usarán para entrenar un modelo de aprendizaje automático, para el desarrollo se generaron nuevas variables derivadas de los datos originales para mejorar la capacidad predictiva, estas características permitieron capturar dinámicas temporales y estructurales relevantes en los mensajes CAN.

Estas características tienen el propósito de mejorar la capacidad predictiva de los modelos, para la primera versión del modelado se implementaron variables derivadas de los campos básicos del mensaje CAN enfocadas en estadísticas simples como sumas, medias y desviaciones estándar, pero se identificaron limitaciones en el rendimiento predictivo de manera que se amplió el conjunto de variables capturando estructuras más robustas mejorando las versiones posteriores al entorno del entrenamiento.

Para concluir, en la versión final se incluyeron operaciones basadas en ventanas móviles (rolling windows), cálculos por grupo y transformaciones categóricas que permitieron representar patrones temporales, frecuencia de aparición, entre otras.

A continuación se presenta el listado de características generadas y agrupadas según su categoría:

Categoría	Nombre de la Variable	Descripción
Estadísticas Básicas	DATA_sum	Suma de los valores de DATA_0_int a DATA_7_int.
	DATA_std	Desviación estándar de los valores de los campos DATA_0_int a DATA_7_int.
	DATA_max / DATA_min	Valor máximo y mínimo entre los campos DATA_0_int a DATA_7_int.
	DATA_mean	Media aritmética de los valores de los campos DATA_0_int a DATA_7_int.
	DATA_range	Rango de los valores DATA_0_int a DATA_7_int (DATA_max - DATA_min).
Diferencias y Deltas	CAN_ID_diff	Diferencia entre valores consecutivos de CAN_ID_dec.
	Timestamp_diff	Diferencia de tiempo entre mensajes consecutivos.

Categoría	Nombre de la Variable	Descripción
	Timestamp_diff_CAN_ID	Diferencia de tiempo entre mensajes con el mismo CAN_ID_dec.
	DATA_i_delta (i = 0 - 7)	Diferencia entre valores sucesivos en cada campo DATA_0_int a DATA_7_int.
Transformaciones Avanzadas	CAN_ID_mod	Resto de CAN_ID_dec dividido entre 100 (puede revelar patrones cíclicos o repetitivos).
	CAN_DLC_enc	Codificación numérica de la combinación CAN_ID_dec + DLC mediante LabelEncoder.
Frecuencias	freq_CAN_ID	Frecuencia total de aparición de cada CAN_ID_dec en el dataset.
	freq_CAN_ID_rolling	Frecuencia de aparición de CAN_ID_dec en una ventana móvil de 10 mensajes.
Rolling Features	DATA_mean_rolling	Promedio móvil de las medias de los valores DATA_0_int a DATA_7_int (ventana de 5 registros).
	DATA_std_rolling	Promedio móvil de las desviaciones estándar de los valores DATA_0_int a DATA_7_int (ventana de 5 registros).
Entropía Aproximada	entropy_proxy	Promedio del número de valores únicos por campo DATA_0_int a DATA_7_int, en una ventana móvil (proxy de entropía).

Categoría	Nombre de la Variable	Descripción
Datos del Dataset	CAN_ID_dec, DLC, DATA_0_int a DATA_7_int	Campos originales del mensaje CAN utilizadas directamente como entrada.

Tabla 8. Descripción de las variables usadas para la ingeniería de características. Fuente: Autor.

En versiones previas, el conjunto de características se limitaba a los siguientes campos CAN_ID_diff, Timestamp_diff, DATA_sum, DATA_std, CAN_ID_mod, DATA_max/ DATA_min, DATA_range, DATA_mean.

b. Targets: Se definieron dos variables objetivo para el desarrollo de cada modelo supervisado que permiten abordar la detección de ataques desde dos diferentes perspectivas, estas son:

- Para la clasificación binaria, la variable objetivo es la columna Ataque que indica si una muestra es un ataque o no mediante las etiquetas de True or False.
- Para la clasificación multiclase, la variable objetivo es la columna Tipo_ataque que indica el tipo específico de ataque, además se implementa la columna Ataque para segmentar a aquellos registros que la columna identifique como positivos (True).

```
# Clasificación binaria variable objetivo
X_bin = df_unificado[features]
y_bin = df_unificado['Ataque']

# Clasificación multiclase variable objetivo
df_multiclase = df_unificado[df_unificado['Ataque'] == True]
X_multi = df_multiclase[features]
y_multi = df_multiclase['Tipo_ataque']
```

Fig 22. Selección de variables objetivo modelo binario y multiclase. Fuente: Autor.

c. División del conjunto de datos: Se realizó la división del conjunto de datos con el propósito de preservar la validez de las evaluaciones, inicialmente se implementaron las verificaciones de la clasificación binaria y la clasificación multiclase mencionado anteriormente en las variables objetivos.

En el enfoque actual se aplicó una separación estratificada que asegura la proporción de clases, en este caso que ataque vs señal normal se mantenga constante tanto en entrenamiento como en prueba, el desarrollo se hizo de la siguiente manera:

- Para la clasificación binaria, se aplicó un split estratificado donde se estipuló el 70% para entrenamiento y el 30% para la prueba, pero manteniendo la proporción original de la clase Ataque.

- Para la clasificación multiclase, se dividió de manera secuencial donde 70% de los primeros registros serían para entrenamiento y el resto para la prueba, esto con el propósito de evitar mezclas que pudieran causar fugas de información temporales.

```
# Split de división estratificada para la clasificación binaria
sss_bin = StratifiedShuffleSplit(n_splits=1, test_size=0.3, random_state=42)
for train_idx, test_idx in sss_bin.split(X_bin, y_bin):
    X_train_bin, X_test_bin = X_bin.iloc[train_idx], X_bin.iloc[test_idx]
    y_train_bin, y_test_bin = y_bin.iloc[train_idx], y_bin.iloc[test_idx]

# Split de división temporal para la clasificación multiclase |
split_idx_multi = int(len(X_multi) * 0.7)
X_train_multi, X_test_multi = X_multi.iloc[:split_idx_multi], X_multi.iloc[split_idx_multi:]
y_train_multi, y_test_multi = y_multi.iloc[:split_idx_multi], y_multi.iloc[split_idx_multi:]
```

Fig 23. División estratificada y temporal para los modelos supervisados. Fuente: Autor

d. Balanceo de clases y Manejo de desequilibrio: Debido a que el conjunto de datos presenta un desequilibrio alto, se aplicaron técnicas específicas para balancear las clases minoritarias y mayoritarias. En este caso, se implementó el resampling en las 3 versiones de desarrollo:

- En el código base se aplicó SMOTE para generar ejemplos sintéticos sobre un subconjunto reducido para la clasificación multiclase.
- En el código con enfoque mejorado se utilizó SMOTEENN, donde se aplicó una combinación de sobremuestreo y submuestreo, además se aplicaron pesos de clase ajustados dinámicamente en modelos como XGBoost, lo que permitió reforzar la robustez frente al desequilibrio.
- En el código final se aplicó SMOTE para generar ejemplos sintéticos únicamente en el conjunto multiclase completo, además se codificaron las etiquetas Tipo_ataque con LabelEncoder para facilitar el modelado y se aplicaron también pesos para cada clase basados en su frecuencia para ser utilizados en el modelo XGBoost con el propósito de contrarrestar el desbalance hacia las clases mayoritarias.

Enfoque	Técnica de Balanceo	Tiempo de Ejecución Aproximado
Código Base	SMOTE (Se aplica sólo sobre el 50% del conjunto de entrenamiento para generar muestras sintéticas.)	13 minutos.
Código con Enfoque Mejorado	SMOTEENN (Sobremuestreo sintético y limpieza con ENN.)	5 horas.

Código Final	SMOTE (Aumenta muestras minoritarias interpolando entre vecinos cercanos.)	47 minutos.
--------------	--	-------------

Tabla 9. Comparación de técnicas de balanceo y tiempos de ejecución. Fuente: Autor.

```
# Balanceo de clases con class_weight clasificación binaria
model_bin = RandomForestClassifier(n_estimators=100, random_state=42, class_weight='balanced')
```

Fig 24. Balanceo de clases clasificación binaria. Fuente: Autor.

```
# Balanceo con SMOTE para el conjunto de entrenamiento multiclase
from imblearn.over_sampling import SMOTE
smote = SMOTE(random_state=42)
X_train_multi_bal, y_train_multi_bal = smote.fit_resample(X_train_multi_scaled, y_train_multi)

# Codificación etiquetas para el conjunto multiclase
label_encoder = LabelEncoder()
y_train_multi_bal_encoded = label_encoder.fit_transform(y_train_multi_bal)
y_test_multi_encoded = label_encoder.transform(y_test_multi)
```

Fig 25. Balanceo SMOTE y codificación de etiquetas para la clasificación multiclase. Fuente: Autor.

```
# Cálculo de pesos con scale_pos_weight para cada clase (XGBoost)
counter = Counter(y_train_multi_bal_encoded)
total = sum(counter.values())
scale_pos_weights = {cls: total / (len(counter) * count) for cls, count in counter.items()}
weights_multi = np.array([scale_pos_weights[cls] for cls in y_train_multi_bal_encoded])
```

Fig 26. Ajuste dinámico de pesos por clase para la clasificación multiclase. Fuente: Autor.

- e. Escalado e imputación:** Se realizó este proceso para preparar los datos antes del entrenamiento con el fin de asegurar que no existieran valores faltantes y que las características estuvieran en la misma escala.

Se implementó la técnica de imputación con la media llamada SimpleImputer, que permite rellenar los valores faltantes presentes en el conjunto de datos, esta decisión se tomó debido a que en ciertas ejecuciones sucesivas del código como interrupciones o errores, se observaba la aparición inesperada de valores nulos, de manera que para evitar estas inconsistencias y asegurar la calidad de los datos en el proceso de entrenamiento, se optó por reemplazar dichos valores con la media de cada característica como se hizo con algunas columnas en la fase de preparación y limpieza.

Por otro lado, se utilizó la herramienta StandardScaler para escalar las variables numéricas, este escalador fue ajustado únicamente con los datos del conjunto de entrenamiento, para posteriormente aplicar también al conjunto de prueba, esto se hizo con el fin de evitar la filtración de información entre los conjuntos y garantizar que los modelos operen con características en la misma escala.

```
# Escalado Random Forest clasificación binaria
scaler_bin = StandardScaler()
X_train_bin_scaled = scaler_bin.fit_transform(X_train_bin)
X_test_bin_scaled = scaler_bin.transform(X_test_bin)
```

Fig 27. Escalado para la clasificación binaria. Fuente: Autor.

```
# Escalado conjunto de entrenamiento multiclase
scaler_multi = StandardScaler()
X_train_multi_scaled = scaler_multi.fit_transform(X_train_multi)

# Imputación para clasificación multiclase
from sklearn.impute import SimpleImputer
imputer = SimpleImputer(strategy='mean')
X_train_multi_scaled = imputer.fit_transform(X_train_multi_scaled)
```

Fig 28. Escalado e imputación para la clasificación multiclase. Fuente: Autor.

A continuación, se presenta una comparativa detallada entre tres versiones de desarrollo, donde se identifican fortalezas y limitaciones de cada implementación en términos de ingeniería de características, manejo de desequilibrios, escalado, técnicas de balanceo, entre otros. También los tipos de modelos abordados (binario, multiclase, híbrido y no supervisado), la siguiente tabla resume aspectos clave para proporcionar una visión estructurada de las decisiones técnicas adoptadas en cada caso.

Dimensiones de Análisis	Código Base	Código con Enfoque Mejorado	Código Final
Ingeniería de Características	Ingeniería básica: Cálculo de diferencias y sumas simples sobre variables DATA_i_int, no incluye features rolling ni estadísticas avanzadas.	Ingeniería intermedia: Incorpora medias móviles y desviaciones estándar rolling, diferencias delta, frecuencia de CAN_ID_dec y codificación de CAN_DLC.	Ingeniería avanzada y optimizada: Incluye estadísticas rolling (media, desviación, rango), diferencias delta, entropía proxy, frecuencia rolling y codificación CAN_DLC.
Variables Objetivo	Clasificación binaria (ataque / no ataque) y multiclase (tipo de ataque).	Igual que en el código base, con selección y filtrado de datos más rigurosos para la clasificación binaria y multiclase.	Clasificación binaria y multiclase, con targets adicionales para evaluación híbrida combinando ambos enfoques.

Dimensiones de Análisis	Código Base	Código con Enfoque Mejorado	Código Final
División de Conjuntos de Datos	División estratificada simple para la clasificación binaria y multiclase utilizando sklearn.	División estratificada para el modelo binario, división temporal (70% - 30%) para el modelo multiclase, evitando la fuga de datos.	Similar al código con enfoque mejorado, con división estratificada y validación cruzada opcional para binario y multiclase.
Balanceo de Clases	SMOTE con reducción del 50% al conjunto de entrenamiento aplicado para el modelo multiclase. Para el modelo binario, ajuste de peso de clases <code>class_weight='balanced'</code> en RandomForest.	SMOTEENN para balanceo avanzado en multiclase, <code>class_weight='balanced'</code> aplicado en el modelo binario.	SMOTE para multiclase, ajuste dinámico de pesos (<code>scale_pos_weight</code>) en XGBoost (binario) para balancear clases.
Manejo de Desequilibrio	Balanceo estándar mediante SMOTE y pesos de clase, sin técnicas adicionales de regularización o detección.	Balanceo mejorado con SMOTEENN, pesos ajustados y técnicas para prevenir sobreajuste.	Balanceo dinámico con pesos ajustados en XGBoost, integración de modelos no supervisados (Isolation Forest, Autoencoder) para detección avanzada de anomalías.
Escalado e Imputación	Escalado estándar con StandardScaler e imputación simple mediante media para valores faltantes.	Escalado con StandardScaler e imputación con SimpleImputer (media), incluye limpieza avanzada previa.	Igual que el código con enfoque mejorado, con pipeline estructurado que realiza imputación posterior al escalado en multiclase.

Dimensiones de Análisis	Código Base	Código con Enfoque Mejorado	Código Final
Enfoques Adicionales	Clasificación básica mediante RandomForest y XGBoost sin ajuste dinámico de umbrales.	Pipeline híbrido con optimización de umbral para binario y multiclase, mejorando precisión y control de falsos positivos.	Pipeline híbrido con umbrales dinámicos y modelos no supervisados complementarios (Isolation Forest, Autoencoder) para detección de anomalías y mejora de robustez.

Tabla 10. Comparativa técnica entre las versiones de modelado. Fuente: Autor.

4.3.2.1 Modelo Supervisado

Se crearon dos modelos principales, los cuales se entrenaron usando etiquetas conocidas de ataques para realizar su correspondiente clasificación, el primer modelo es binario que se implementó para detectar si un registro es ataque o no y el segundo modelo es multiclase que permite clasificar el tipo específico de ataque siempre y cuando sea un ataque, en ambas clasificaciones se aplicaron técnicas de escalado, balanceo y ajustes de hiperparámetros para mejorar su rendimiento.

- a. **Modelo Binario:** Este clasificador busca predecir si un registro es un ataque o un tráfico normal, implementando Random Forest para su desarrollo debido a la robustez frente a datos heterogéneos y la capacidad que tiene para manejar desequilibrios en las clases mediante la ponderación. Por otro lado, se implementa como un filtro inicial en el sistema para identificar señales maliciosas para posteriormente poder analizarlos con más detalle en los siguientes modelos.

```
# Escalado Random Forest clasificación binaria
scaler_bin = StandardScaler()
X_train_bin_scaled = scaler_bin.fit_transform(X_train_bin)
X_test_bin_scaled = scaler_bin.transform(X_test_bin)

# Balanceo de clases con class_weight clasificación binaria
model_bin = RandomForestClassifier(n_estimators=100, random_state=42, class_weight='balanced')

#Entrenamiento del modelo binario
model_bin.fit(X_train_bin_scaled, y_train_bin)
y_pred_bin = model_bin.predict(X_test_bin_scaled)
```

Fig 29. Creación y entrenamiento del modelo binario. Fuente: Autor.

- b. **Modelo Multiclase:** Este clasificador busca identificar el tipo de ataque que se está presentando en un registro, implementando un modelo XGBoost que utiliza técnicas avanzadas de ajuste de pesos para cada clase y mejora la capacidad de distinguir entre los diferentes tipos de ataques lo cual aporta detalle a la detección.

```
# Inicialización del modelo XGBoost con hiperparámetros mejorados
model_multi = XGBClassifier( n_estimators=500, max_depth=8, learning_rate=0.05, subsample=0.8, colsample_bytree=0.8,
                             objective='multi:softprob',
                             num_class=len(label_encoder.classes_),
                             eval_metric='mlogloss',
                             use_label_encoder=False,
                             random_state=42
                           )

# Validación cruzada
cv = StratifiedKFold(n_splits=3, shuffle=True, random_state=42)
# scores = cross_val_score(model_multi, X_train_multi_bal, y_train_multi_bal_encoded,
#                           scoring='f1_weighted', cv=cv, fit_params={'sample_weight': weights_multi})
# print(f"F1 promedio (CV 3-fold): {scores.mean():.4f}")

# Entrenamiento final con pesos para balancear clases
model_multi.fit(X_train_multi_bal, y_train_multi_bal_encoded, sample_weight=weights_multi)
```

Fig 30. Creación y entrenamiento del modelo multiclase. Fuente: Autor.

Durante esta fase se eliminó la variable Timestamp de los features utilizados en los modelos, dado que su inclusión inducía sesgos en la clasificación. En este caso, se incorporaron características derivadas de la variabilidad informativa donde se destacó la variable entropy_proxy, que es una métrica proxy de entropía calculada mediante el promedio rodante del número de valores únicos en las variables DATA_0_int a DATA_7_int, lo cual resultó siendo la más relevante para la determinación del estado de ataque.

Inicialmente, el modelo presentaba problemas significativos de confusión entre tipos de ataques, especialmente entre clases similares. Para mitigar este problema y mejorar el desempeño, se implementaron varias técnicas clave:

- **Balanceo de clases mediante SMOTE:** Para el escenario multiclase, se aplicó la técnica de sobremuestreo sintético SMOTE, que genera instancias artificiales de las clases minoritarias, equilibrando así la distribución de clases y facilitando un aprendizaje más equitativo.
- **Validación cruzada estratificada:** Para evaluar la capacidad de generalización y evitar sesgos de muestreo, se utilizó StratifiedKFold, que preserva la proporción de clases en cada partición durante el proceso de validación cruzada.
- **Ajuste de hiperparámetros:** El modelo multiclase XGBoost fue configurado con hiperparámetros optimizados para mejorar el rendimiento, incluyendo un número elevado de estimadores ($n_estimators = 500$), profundidad máxima ($max_depth = 8$), tasa de aprendizaje moderada ($learning_rate = 0.05$), y muestreos ($subsample = 0.8$, $colsample_bytree = 0.8$).

- **Métricas detalladas para evaluación:** Se utilizaron métricas como el F1-score ponderado, precisión y recall para obtener una visión integral del comportamiento del modelo, especialmente en escenarios con clases desbalanceadas.

4.3.2.2 Modelo No Supervisado

Se implementaron dos técnicas que modelan el comportamiento normal y detección de desviaciones para detectar anomalías dentro del tráfico, sin requerir a etiquetas previas o explícitas lo que permite identificar ataques nuevos o desconocidos.

- Modelo Isolation Forest:** Este método detecta anomalías aislando observaciones en particiones aleatorias, este modelo se entrenó exclusivamente con muestras normales para aprender su distribución y posteriormente poder detectar las desviaciones en el conjunto de test como posibles ataques.

```
# Inicialización y entrenamiento el modelo Isolation Forest
iso_model = IsolationForest(n_estimators=200, contamination=0.2, random_state=42)
X_train_normales = X_train_bin_scaled[y_train_bin == False]
iso_model.fit(X_train_normales)
```

Fig 31. Creación y entrenamiento del modelo Isolation Forest. Fuente: Autor

- Modelo Autoencoder Denso:** Es un tipo de red neuronal que aprende a reconstruir señales normales con un bajo error, el cual especifica que si al reconstruir una señal el error es muy grande puede significar que se hallar una señal de anomalía.

```
# Inicialización del modelo Autoencoder Denso
input_layer = Input(shape=(input_dim,))
encoded = Dense(encoding_dim, activation="relu")(input_layer)
decoded = Dense(input_dim, activation="linear")(encoded)
autoencoder = Model(inputs=input_layer, outputs=decoded)

# Entrenamando del modelo Autoencoder Denso
autoencoder.compile(optimizer='adam', loss='mse')
early_stopping = EarlyStopping(monitor='val_loss', patience=5, restore_best_weights=True)

autoencoder.fit(X_train_autoenc, X_train_autoenc, epochs=50, batch_size=64, validation_split=0.2, callbacks=[early_stopping],
               verbose=1)
```

Fig 32. Creación y entrenamiento del modelo Autoencoder Denso. Fuente: Autor.

4.3.2.3 Pipeline Híbrido

Finalmente, se desarrolló un Pipeline que combina los modelos supervisados con la sensibilidad a anomalías de los modelos no supervisados con el propósito de obtener una detección más robusta y confiable. La lógica con la que se desarrolló fue llamar al modelo binario y aquellas muestras sospechosas son clasificadas en detalle con el modelo multiclase y si se encuentran registros con baja confianza se clasifican como desconocido, por último en este proceso los modelos no supervisados actúan como una capa adicional de alerta para detectar patrones novedosos.

Para el desarrollo, se emplea un umbral óptimo determinado a partir de la curva ROC para tomar decisiones en la clasificación binaria, para la clasificación multiclase se utiliza un enfoque de confianza dinámica que considera la probabilidad más alta de predicción para aceptar o rechazar un resultado, todas las predicciones se integran en un único DataFrame, lo que permite un análisis más organizado y eficiente, para finalmente, realizar una evaluación comparativa del pipeline híbrido propuesto frente a modelos individuales, con el fin de medir su desempeño relativo.

```
# Pipeline Híbrido

# Predecir probabilidades para el conjunto de test binario
y_prob_bin = model_bin.predict_proba(X_test_bin_scaled)[: , 1]

# Umbral dinámico óptimo
fpr, tpr, thresholds = roc_curve(y_test_bin, y_prob_bin)
optimal_idx = np.argmax(tpr - fpr)
threshold_conf = thresholds[optimal_idx]

# Aplicar el umbral al modelo binario
y_pred_bin_thresh = y_prob_bin > threshold_conf

# Crear un DataFrame con los datos necesarios
df_test = X_test_bin.copy()
df_test['pred_bin_thresh'] = y_pred_bin_thresh
df_test['true_label'] = y_test_multi.reindex(X_test_bin.index, fill_value='desconocido')

# Identificar los índices de muestras clasificadas como ataque
idx_sospechosos = df_test[df_test['pred_bin_thresh']].index
idx_normales = df_test[~df_test['pred_bin_thresh']].index

# Para muestras normales, predecir directamente como 'señal_normal'
df_test.loc[idx_normales, 'pred_hybrid'] = 'señal_normal'
df_test.loc[idx_normales, 'true_label'] = 'señal_normal'

# Procesar los sospechosos en lote
X_sospechosos_scaled = scaler_multi.transform(X_test_bin.loc[idx_sospechosos, features].values)
probas_sospechosos = model_multi.predict_proba(X_sospechosos_scaled)
max_probas = np.max(probabilities_sospechosos, axis=1)
pred_encoded = np.argmax(probabilities_sospechosos, axis=1)
pred_labels = label_encoder.inverse_transform(pred_encoded)

# Aplicar lógica de confianza
pred_final = np.where(max_probas < threshold_conf, 'desconocido', pred_labels)

# Asignar predicciones al DataFrame
df_test.loc[idx_sospechosos, 'pred_hybrid'] = pred_final

# Resultado final
y_pred_hybrid = df_test['pred_hybrid'].values
y_true_hybrid = df_test['true_label'].values
```

Fig 33. Implementación de pipeline híbrido con lógica de umbral dinámico. Fuente: Autor.

A continuación, se presenta una tabla comparativa entre las versiones implementadas para los modelos con el fin de resaltar las diferencias y mejoras progresivas entre cada versión implementada.

Modelos	Código Base	Código con Enfoque Mejorado	Código Final
Clasificación Binaria	Random Forest básico con 100 árboles y balanceo de clases.	Igual que en el código base, pero Random Forest con balanceo).	Igual que el código con enfoque mejorado.
Clasificación Multiclase	XGBoost con 200 árboles, configuración básica y ajuste manual del peso por clase.	XGBoost mejorado con 500 árboles, profundidad 8, tasa de aprendizaje baja, con técnicas para reducir sobreajuste y balanceo avanzado (SMOTEENN).	XGBoost con parámetros similares al código con enfoque mejorado, pero con cálculo automático de pesos para balancear las clases.
Pipeline Híbrido de Clasificación	Umbral fijo para distinguir entre normal y ataque antes de clasificar tipos de ataque.	Umbral óptimo calculado automáticamente mediante curvas ROC, con lógica más eficiente para clasificación híbrida.	Pipeline híbrido con umbral dinámico y asignación de etiquetas 'desconocido' para casos poco confiables mejorando la precisión.
Isolation Forest	Parámetros básicos, contaminación fija.	Similar al código base, pero con un ligero ajuste para capturar más anomalías.	Mismo enfoque del código mejorado, con ajuste fino de contaminación para mejorar detección.
Autoencoder	Autoencoder LSTM para reconstrucción.	Autoencoder denso para mayor eficiencia computacional.	Autoencoder denso con submuestreo y umbral optimizado basado en curvas ROC para mejor precisión.

Tabla 11. Resumen comparativo del desarrollo de modelos según versión de desarrollo. Fuente: Autor.

4.3.3 Visualizaciones

En este apartado se comparan los modelos supervisados y no supervisados con base en métricas de rendimiento y características del conjunto de datos con el objetivo de identificar las metodologías más eficaces para la prevención de ataques y contribuir al desarrollo de defensas robustas en el ámbito de la ciberseguridad vehicular.

En el siguiente mapa de calor, se puede identificar relaciones lineales entre las variables extraídas del bus CAN, donde presenta una alta correlación positiva entre múltiples características con campos como DATA_DATA_0_int a DATA_7_int y sus versiones delta (DATA_x_delta), DATA_sum, DATA_mean, DATA_range, entre otras. Esto sugiere una estructura redundante en los datos del bus CAN que debe ser considerada para evitar sobre ajustes y mejorar la generalización del modelo.

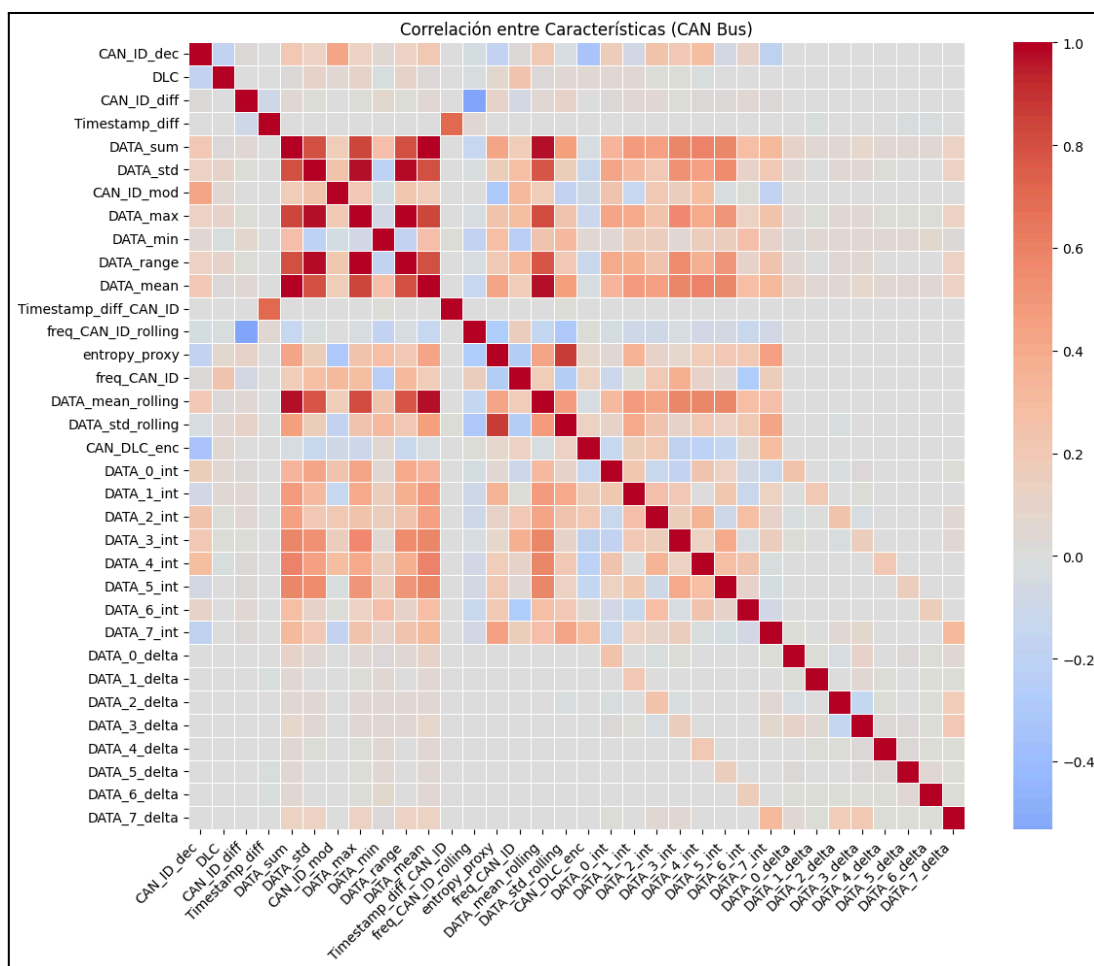


Fig 34. Mapa de calor de correlación entre variables del bus CAN. Fuente: Autor.

En el modelo binario, la característica más relevante es DATA_min, seguida por DATA_std, DATA_range y valores individuales de los bytes del campo DATA_0_int a DATA_7_int, lo cual indica que las variaciones estadísticas y las transformaciones simples del contenido referente al mensaje CAN son altamente predictivas para distinguir entre un tráfico normal y ataques.

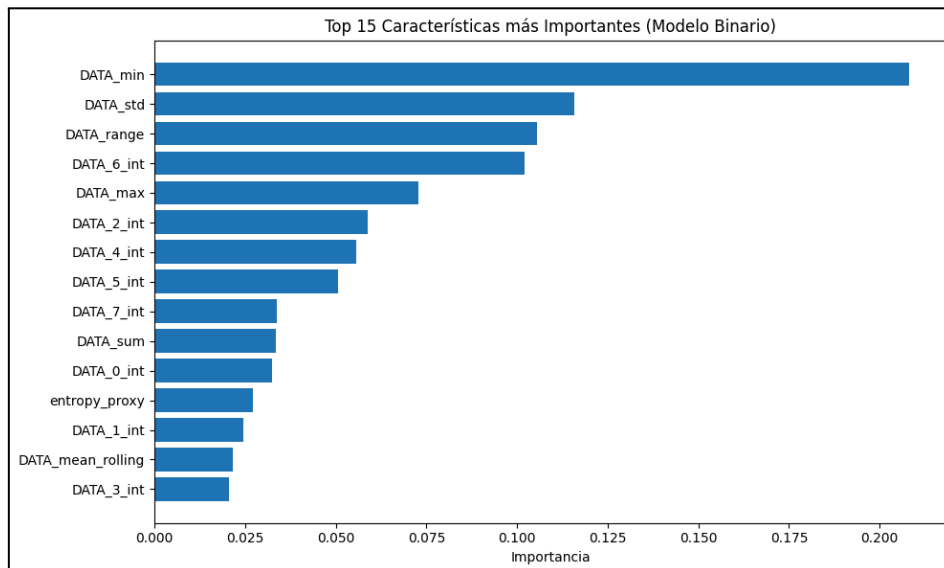


Fig 35. Ranking de las 15 características más relevantes según el modelo binario. Fuente: Autor.

Para evaluar cada modelo, se utilizaron las métricas básicas de evaluación como Precisión (mide la exactitud del modelo al predecir la clase positiva), Recall (evalúa la capacidad del modelo para identificar correctamente los casos positivos), F1-Score (métrica de equilibrio cuando se desea considerar tanto la precisión como la cobertura del modelo) y MCC (mide la calidad de clasificación binaria que considera todas las combinaciones de verdaderos, falsos positivos y negativos) en diferentes enfoques de modelado.

Según los resultados, se evidencia que los modelos Binario y Autoencoder Denso muestran el mejor rendimiento, a diferencia del modelo Multiclase que es el peor, con métricas muy bajas, especialmente en el MCC. Por otra parte, el Pipeline Híbrido ofrece un desempeño equilibrado y el modelo Isolation Forest tiene alta precisión pero bajo recall y MCC, lo que indica problemas para detectar positivos.

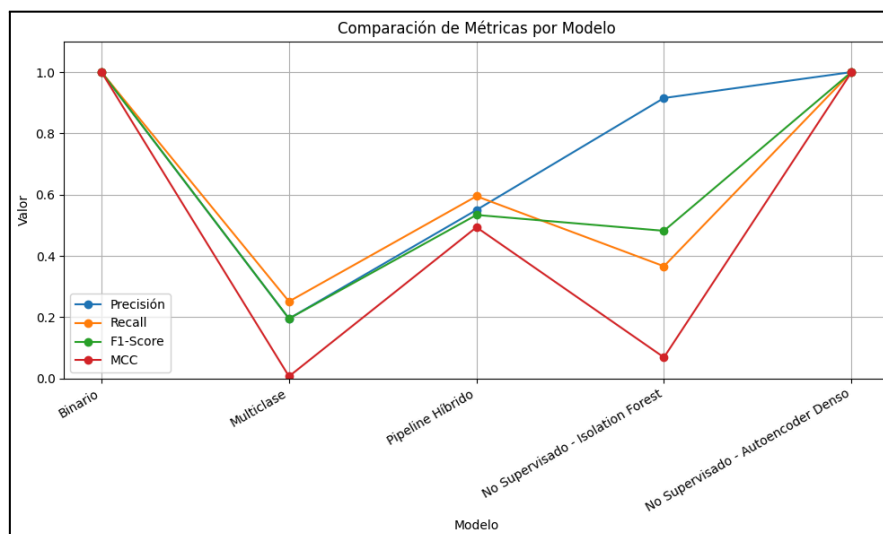


Fig 36. Comparación de métricas de desempeño entre modelos de detección. Fuente: Autor.

El gráfico de radar ofrece una visualización integral del rendimiento de cada modelo según diferentes métricas, en este se puede apreciar que el modelo Autoencoder Denso domina en todas las dimensiones evaluadas siendo una solución no supervisada sumamente competitiva, seguido por el modelo binario y el pipeline híbrido, nuevamente el modelo multiclase se diferencia siendo el más débil en todas las métricas.

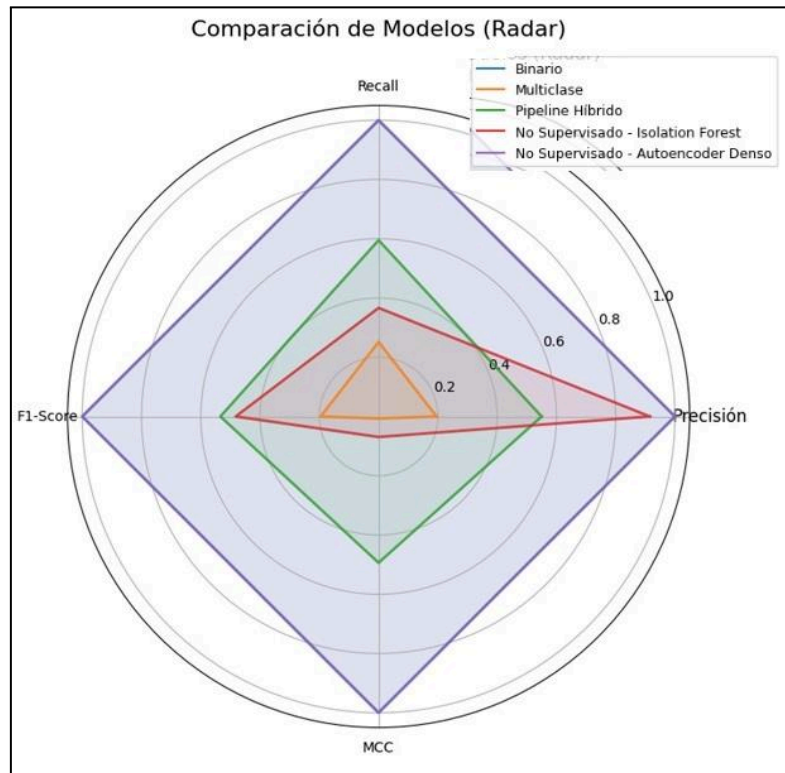


Fig 37. Comparación integral del desempeño de modelos de detección. Fuente: Autor.

La matriz de confusión del modelo binario muestra una clasificación perfecta tanto para tráfico normal como para ataques como se muestra con el porcentaje del 100% en aciertos, en este caso sugiere que es una excelente opción cuando se dispone de etiquetas claras, pero posiblemente podría reflejar un sobreajuste o un conjunto de datos muy bien definido.

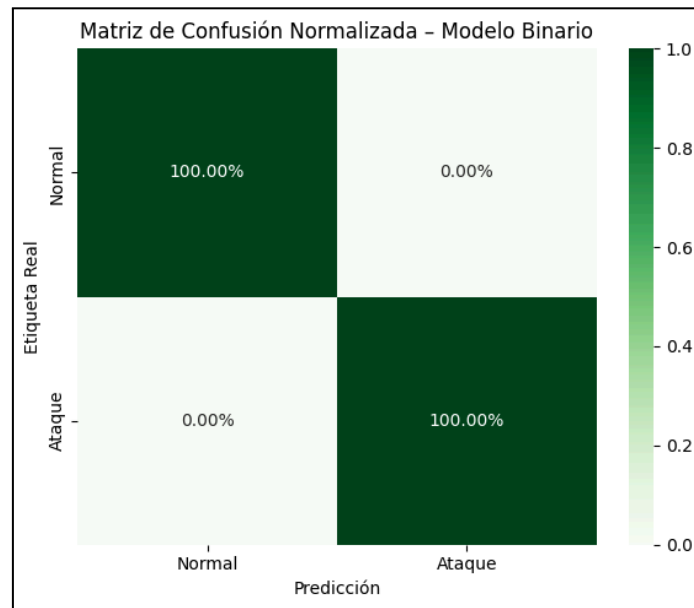


Fig 38. Desempeño del modelo binario en detección de ataques. Fuente: Autor.

Este modelo no supervisado alcanza un rendimiento similarmente perfecto con un porcentaje de 99.96% y 99.99% para cada clase de ataque, por lo tanto, el modelo Autoencoder Denso demuestra ser una herramienta eficaz para la detección de anomalías sin supervisión.

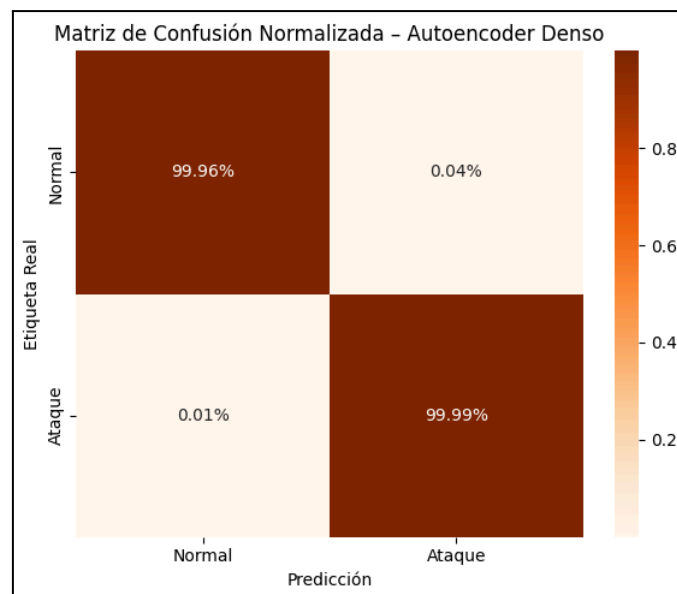


Fig 39. Desempeño del modelo Autoencoder Denso en detección de ataques. Fuente: Autor.

El modelo multiclase presente en el pipeline híbrido puede identificar algunos tipos de ataques con buena precisión, aunque su capacidad para distinguir entre los diferentes tipos de ataques aún es limitada, lo cual puede dificultar respuestas específicas ante amenazas.

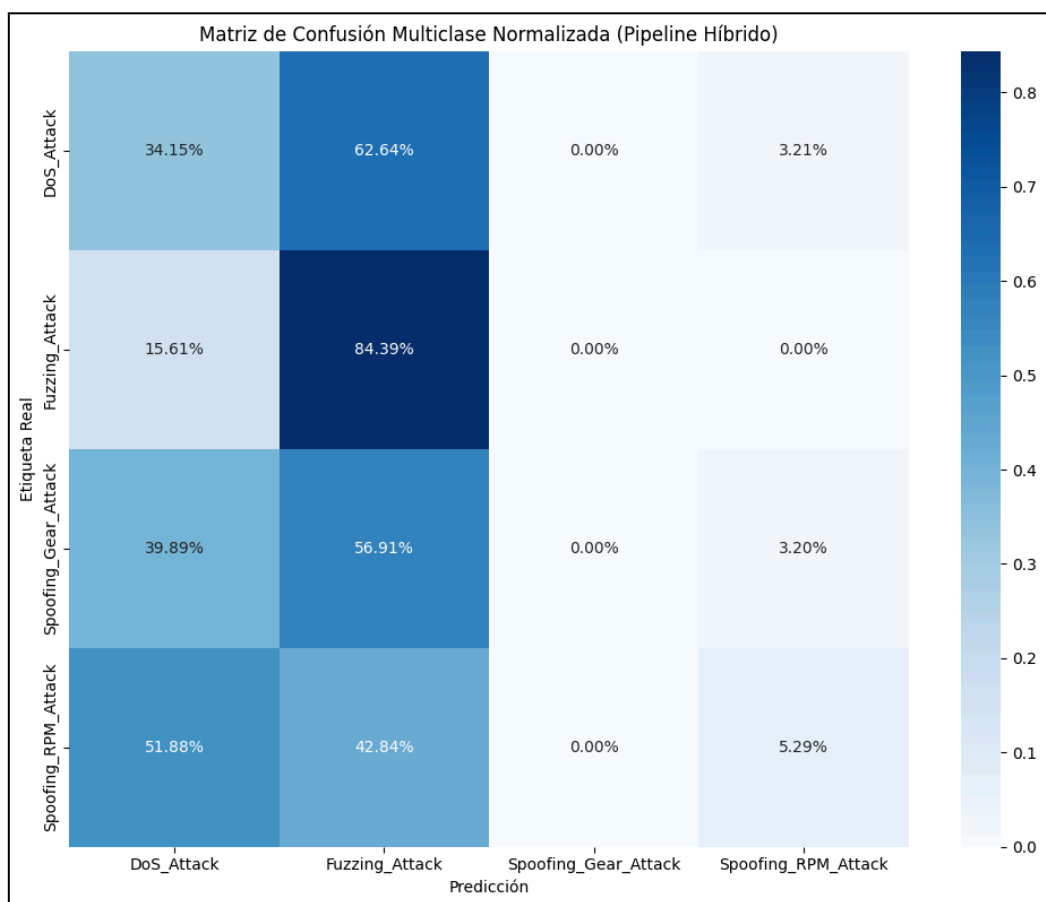


Fig 40. Matriz de confusión normalizada del modelo pipeline híbrido multiclase. Fuente: Autor.

Los resultados presentados indican que tanto los modelos supervisados como no supervisados son efectivos para detectar ciberataques en datos simulados del bus CAN, donde el modelo binario y el Autoencoder Denso presentan un rendimiento excelente, lo que respalda su aplicación en sistemas de detección en tiempo real. Por otro lado, el modelo multiclase requiere mejoras, posiblemente en términos de ingeniería de características o balance de clases para lograr diferenciar efectivamente los tipos de ataque.

A continuación, se presenta una comparación de desempeño entre distintos enfoques de modelos, evaluados en tres versiones con diferentes configuraciones. Se incluyen métricas clave como Precisión, Recall, F1-Score y MCC para cada modelo, lo que permite identificar cuál de ellos ofrece el mejor equilibrio entre eficacia y eficiencia en contextos prácticos.

Modelo	Versión	Precisión	Recall	F1-Score	MCC	Observaciones
Binario	Código Base	1.000	1.000	1.000	1.000	Resultados perfectos.

Modelo	Versión	Precisión	Recall	F1-Score	MCC	Observaciones
	Código con Enfoque Mejorado	1.000	1.000	1.000	1.000	Resultados perfectos.
	Código Final	1.000	1.000	1.000	1.000	Resultados perfectos.
Multiclase	Código Base	0.210	0.272	0.223	0.020	Bajo rendimiento.
	Código con Enfoque Mejorado	0.305	0.271	0.236	0.048	Mejora leve.
	Código Final	0.195	0.251	0.195	0.007	Peor MCC.
Pipeline Híbrido	Código Base	0.730	0.414	0.484	0.265	Buen balance entre precisión y recall.
	Código con Enfoque Mejorado	0.575	0.586	0.576	0.459	Mejora general (mejor F1 y MCC).
	Código Final	0.550	0.594	0.533	0.493	Mejor MCC.
Isolation Forest	Código Base	0.918	0.394	0.513	0.082	Recall bajo.
	Código con Enfoque Mejorado	0.915	0.366	0.482	0.069	Leve disminución.
	Código Final	0.915	0.366	0.482	0.069	Mismos resultados que en código con enfoque mejorado.

Modelo	Versión	Precisión	Recall	F1-Score	MCC	Observaciones
Autoencoder Denso	Código Base	0.999	0.999	0.999	0.999	Casi perfecto.
	Código con Enfoque Mejorado	0.999	0.999	0.999	0.999	Mínima diferencia.
	Código Final	0.999	0.999	0.999	0.999	Ligera caída.

Tabla 12. Comparación de métricas de rendimiento de modelos de detección. Fuente: Autor.

A pesar de que el código con enfoque mejorado mostró ligeramente un mejor desempeño en la detección híbrida de ataques, su tiempo de ejecución fue excesivo (más de 5 horas solo en la etapa de balanceo multiclase), lo cual limita su uso en sistemas con recursos limitados. Por esta razón, se considera que el código final es el más adecuado, ya que logra un rendimiento sólido con un tiempo de procesamiento mucho más razonable. Esto lo convierte en una opción más práctica para su implementación en sistemas de detección en tiempo real.

5. Recomendaciones para la Mejora de la Ciberseguridad Vehicular

Según los resultados obtenidos dentro del desarrollo del proyecto desde el análisis y la aplicación de técnicas de aprendizaje automático, se encontraron diversas oportunidades para mejorar la detección y prevención de ciberataques en vehículos conectados.

A continuación se presentan estrategias fundamentadas en los hallazgos obtenidos que están orientadas a fortalecer la seguridad mediante un enfoque integral y adaptable a distintas amenazas.

- Se recomienda utilizar un modelo supervisado como el clasificador binario para detectar ataques conocidos, y complementarlo con un modelo no supervisado como el autoencoder que identifique comportamientos anómalos no previamente etiquetados, esto permitiría ampliar la cobertura del sistema ante tanto ataques conocidos como emergentes.
- Dado que ciertas variables como DATA_min, DATA_std, DATA_range y segmentos específicos de los datos CAN fueron altamente relevantes para la detección, se sugiere realizar monitoreos en estas características, con el fin de optimizar la detección en tiempo real usando sólo las variables más relevantes, mejorando la velocidad y eficiencia del sistema.

- El análisis de correlación entre características revela redundancia entre múltiples variables, por lo tanto se considera beneficiosa la aplicación de técnicas de reducción de dimensionalidad o selección de características haciendo el sistema más liviano y eficiente de implementar en dispositivos embarcados dentro del vehículo.
- Considerando la naturaleza cambiante de las amenazas cibernéticas, se propone proveer al sistema de mecanismos de aprendizaje continuo con modelos que puedan adaptarse y actualizarse periódicamente, esto puede lograrse mediante la incorporación de modelos no supervisados con capacidad de adaptación y reentrenamiento periódico, asegurando una respuesta efectiva ante nuevas variantes de ataque sin necesidad de rediseñar todo el modelo.

6. Conclusiones

1. De acuerdo con el procesamiento y limpieza planteado se logró consolidar y unificar exitosamente cinco archivos heterogéneos, definiendo claramente cada campo para garantizar la consistencia y calidad de los registros. Asimismo, se enriqueció el conjunto de datos mediante la incorporación de campos de etiquetas y la transformación de valores de formato hexadecimal a decimal, lo que permitió estandarizar la información y asegurar su compatibilidad con los métodos de análisis posteriores, estableciendo así una base sólida y confiable para las etapas siguientes del estudio.
2. Se desarrolló un análisis exploratorio que permitió identificar características clave del dataset, como la presencia y distribución de valores nulos, la cantidad de valores únicos por variable y patrones relevantes en las señales vehiculares mediante visualizaciones descriptivas. Este proceso facilitó la detección de un desequilibrio significativo en las clases, con predominancia de ataques Spoofing (RPM y Gear) frente al tráfico legítimo, y evidenció que ciertos identificadores CAN están asociados a tipos específicos de ataques. Por otro lado, se observaron diferencias claras en la variabilidad y entropía entre señales manipuladas y legítimas, aspectos que pueden ser aprovechados para mejorar la detección de anomalías.
3. De acuerdo con los resultados obtenidos en la fase de modelado demuestran que los modelos de clasificación binaria y el autoencoder denso lograron un desempeño sobresaliente en la detección de ataques en el tráfico CAN, alcanzando métricas cercanas al 100% en precisión, recall, F1-score y MCC, esto evidencia la alta efectividad de combinar técnicas supervisadas y no supervisadas para identificar comportamientos anómalos; Sin embargo, el modelo multiclase presentó limitaciones para diferenciar con precisión entre tipos específicos de ataques como

DoS, fuzzing y spoofing, lo que indica que para mejorar la clasificación detallada se requiere una mayor sofisticación en el preprocesamiento, la selección de características o el uso de modelos más avanzados.

4. En el transcurso del desarrollo de los distintos pipelines, se identificó que el enfoque basado en la técnica de balanceo SMOTEENN con pesos dinámicos, aunque ofreció un desempeño destacado, resultó poco práctico debido a un tiempo de procesamiento, a comparación de la aplicación de SMOTE junto con una estrategia de pesos basada en la frecuencia de las clases que logró mantener un buen rendimiento reduciendo significativamente el tiempo de ejecución, dando un enfoque de balance más adecuado entre precisión y eficiencia computacional para su aplicación en escenarios reales.
5. El estudio llevado a cabo confirma que la combinación de modelos supervisados y no supervisados mejora la detección temprana de ciberataques en el bus CAN, además de la identificación de variables clave que optimizan la detección en tiempo real. De acuerdo a lo anterior, se recomienda la incorporación de mecanismos de aprendizaje continuo para que los sistemas puedan adaptarse a nuevas amenazas sin necesidad de rediseños completos. Asimismo, se aconseja implementar técnicas de reducción de dimensionalidad para facilitar la integración en entornos con recursos limitados, y continuar optimizando el equilibrio entre precisión y eficiencia computacional para garantizar sistemas de ciberseguridad robustos y prácticos.

7. Recomendaciones y Prospectiva

A partir del análisis comparativo de las tres versiones del pipeline de detección de ataques vehiculares (códigos base, mejorado y final), se establecen las siguientes recomendaciones clave para su implementación práctica:

- a. El código final representa la opción más equilibrada en términos de rendimiento y eficiencia, a pesar de no obtener la métrica más alta en F1-score, su F1 híbrido de 0.53 junto con un MCC de 0.49 y un tiempo de ejecución razonable, lo convierten en la alternativa más viable para aplicaciones reales.
- b. El código con enfoque mejorado, aunque obtuvo el mejor F1 híbrido (0.57), implicó un tiempo de ejecución exagerado, lo que limita su uso en entornos productivos donde los recursos computacionales o el tiempo de procesamiento son críticos..

- c. El código base, con una ejecución significativamente más rápida y resultados aceptables ($F1: 0.48$), es recomendable en fases iniciales de desarrollo o para entornos con hardware limitado.
- d. En todas las versiones, el Autoencoder demostró un rendimiento sobresaliente ($F1 > 0.99$), lo que lo posiciona como una herramienta sólida para la detección de anomalías en escenarios donde no se cuenta con etiquetas. De igual forma el modelo binario, permitiendo usarlo como una fase inicial en los sistemas para identificación de ataques positivos.

Por otra parte, a lo largo del desarrollo del proyecto se identificaron diversas áreas con el potencial para ser optimizadas o exploradas en futuras etapas, de esta manera se presentan algunas recomendaciones con prospectiva orientadas a fortalecer la eficacia, eficiencia y escalabilidad del sistema propuesto:

- Considerando los largos tiempos de entrenamiento observados especialmente en la versión mejorada, se recomienda evaluar métodos de balanceo más rápidos o parciales que reduzcan la carga computacional sin comprometer la calidad de los datos.
- Dado a los bajos resultados obtenidos en la clasificación multiclase, los cuales evidencian la necesidad de contar con más datos etiquetados, especialmente para clases de ataque con baja representación, se recomienda enriquecer el dataset mediante simulaciones realistas adicionales o técnicas de data augmentation.
- Otra recomendación frente a las limitaciones observadas en el modelo multiclase para distinguir entre tipos de ataque, es que se sugiere entrenar un modelo distinto para cada tipo de ataque para tener una mejor precisión al identificar qué tipo de ataque está ocurriendo, y así aplicar medidas de respuesta más adecuadas.
- Para mejorar la detección de patrones temporales complejos se plantea la posibilidad de integrar arquitecturas más robustas, siempre y cuando se controle el uso de memoria y tiempo de cómputo.
- El alto rendimiento del modelo binario y del autoencoder denso sugiere que estos enfoques podrían ser integrados en sistemas reales como unidades de control vehicular. No obstante, se recomienda aplicar optimizaciones adicionales que reduzcan el consumo de recursos, como poda de árboles o cuantización de pesos en modelos neuronales.
- Se recomienda investigar y seleccionar características específicas que distingan mejor entre ataques de tipo spoofing, DoS y fuzzing, esto podría mejorar significativamente el rendimiento de modelos multicategoricos, al proveer señales más discriminativas para cada clase.

8. Referencia Bibliográfica

Cañones, D & Santos, J (2019). Lenguajes de Programacion. CEUPE

Cañones, D & Santos, J (2019). Estudio y Modelado del Dato. CEUPE

Cañones, D & Santos, J (2019). Diseño de un modelo escalable. CEUPE

Cañones, D & Santos, J (2019). Herramientas y Librerías. CEUPE

[4] Fortra (2020, 23 julio). Fuzzing Penetration Testing: An Important Tool for Cybersecurity. Sitio Web: <https://www.fortra.com/resources/guides/fuzzing-in-cybersecurity>

[5] Pearson, J. (2022, 17 de mayo). Tesla cars, Bluetooth locks, vulnerable to hackers, researchers say. Reuters. Sitio Web: <https://www.reuters.com/technology/tesla-cars-bluetooth-locks-vulnerable-hackers-researchers-2022-05-17/>

Hispasec. (2025, 6 de mayo). Vulnerabilidad crítica en Tesla Model 3 a través del TPMS. Una al día. Sitio Web: <https://unaaldia.hispasec.com/2025/05/vulnerabilidad-critica-en-tesla-model-3-a-traves-del-tpms.html>

Zero Day Initiative. (2023, 14 de septiembre). ZDI-23-972: Tesla Infotainment Linux Kernel Raw CAN Frame Privilege Escalation Vulnerability. Sitio Web: <https://www.zerodayinitiative.com/advisories/ZDI-23-972/>

[8] Choi, J. (2018). The Car Hacking Dataset, HKSecurity OCSLab. Sitio Web: <https://ocslab.hksecurity.net/Datasets/car-hacking-dataset>

Texas Instruments. (2016). Introduction to the Controller Area Network (CAN) (Rev. B). Application report. <https://www.ti.com/lit/an/sloa101b/sloa101b.pdf>

Spark Apache. (s. f.). Getting Started – Spark SQL, DataFrames and Datasets Guide. Sitio web: <https://spark.apache.org/docs/latest/sql-getting-started.html>

Documento consultado MDPI, <https://www.mdpi.com/1099-4300/26/12/1126>. Fecha de consulta: 10 de agosto de 2025.

Documento consultado UpGrad, <https://www.upgrad.com/tutorials/ai-ml/machine-learning-tutorial/entropy-in-machine-learning>.

Fecha de consulta: 10 de agosto de 2025.

Towards Data Science, 2019, Applications of Rolling Windows for Time Series with Python. Disponible en:

<https://towardsdatascience.com/applications-of-rolling-windows-for-time-series-with-python-1a4bbe44901d/> . Fecha de consulta: 14 de agosto de 2025.

KDNuggets, 2017, *Automated Feature Engineering for Time Series Data*. Disponible en: <https://www.kdnuggets.com/2017/11/automated-feature-engineering-time-series-data.html>. Fecha de consulta: 14 de agosto de 2025.

KDNuggets, 2018, *Feature Engineering Explained with 10 Examples*. Disponible en: <https://www.kdnuggets.com/2018/12/feature-engineering-explained.html>. Fecha de consulta: 14 de agosto de 2025.

dotData, 2021, Practical Guide for Feature Engineering of Time Series Data. Disponible en: <https://dotdata.com/blog/practical-guide-for-feature-engineering-of-time-series-data>. Fecha de consulta: 15 de agosto de 2025.

Scikit-learn, s.f. Metrics and scoring: quantifying the quality of predictions. Disponible en: https://scikit-learn.org/stable/modules/model_evaluation.html. Fecha de consulta: 19 de agosto de 2025.

OpenAI. ChatGPT (modelo GPT-4). Herramienta de lenguaje natural basada en IA. Disponible en: <https://chat.openai.com>.